Software Engineering

Chapter 5: Software Design, Architecture & OO Principles

From Universal Foundations & Architecture Modeling to SOLID Craftsmanship

Prof. Nien-Lin Hsueh
Department of Information Engineering and Computer Science
Feng Chia University

Chapter 5: Roadmap & Core Curriculum

Part 1: Process, Principles & Macro Design

- **5.1 Software Design Foundations & Process:** The 4-stage design process (Arch → Interface → OO → Data) and specifications (SDD & ADRs).
- **5.2 Universal Design Principles:** Abstraction, encapsulation, separation of concerns, modularity, and cohesion & coupling.
- **5.3 Stage 1 — Architectural Design:** 6 classic patterns, Food Delivery synthesis, ATAM, and **UML Package & Deployment Diagrams**.
- **5.4 Stage 2 — Interface & Contract Spec: Design by Contract (DbC)** and **UML Component Diagrams** (Lollipop & Socket).

Part 2: Micro OO Design, Data, AI & Synthesis

- **5.5 Stage 3 — Component & OO Design:** GoF 23 Design Patterns and the 5 SOLID principles via **UML Class Diagrams**.
- **5.6 Stage 4 — Data & Persistence Design:** 3-level data modeling, **UML ERD**, 3NF, Redis Geo-caching, and polyglot persistence.
- **5.7 AI-Augmented Software Design:** AI copilots across all 4 design stages and human-in-the-loop blind spot governance.
- **5.8 Conceptual Synthesis & References:** End-to-end curriculum review, self-assessment matrix, and classic literature.

5.1 Software Design Foundations, Process & Specifications

> *"There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies."*
> *— C. A. R. Hoare (Turing Award Laureate)*

What is Software Design?

- **From Requirements (What) to Implementation (How):**
  - **Requirements Engineering (SRS)**: Defines stakeholder problems, use cases, and quality expectations (*what to build*).
  - **Software Design (SDD)**: Conceptualizes structural decomposition, interface contracts, class hierarchies, and data schemas (*how to build*).
- **Core Activities of Software Design:**
  - **Blueprint & Specification Creation**: Establishes architectural models, formal interface contracts, and database schemas so teams can build in parallel.
  - **Trade-off Decision Making**: Balances competing non-functional forces (e.g., latency vs. consistency vs. cloud cost).
- **The Distinction Between Design and Coding:**
  - Coding expresses logic in a specific programming language syntax.
  - Design determines **how modules are partitioned, how interfaces are contracted, and how data is persisted** for long-term evolvability.

Why is Software Design Crucial?

- **1. Eliminating "Vibe Coding" and Structural Code Rot:**
  - Coding without design rapidly triggers the **4 Symptoms of Poor Design** (Robert C. Martin):
    - **Rigidity**: A single change forces cascading edits across dozens of unrelated files.
    - **Fragility**: Fixing a bug in Module A inexplicably breaks an unrelated Module B.
    - **Immobility**: Tangled components cannot be extracted for reuse in other systems.
    - **Viscosity**: Doing things right is harder than hacking, tempting devs to take shortcuts.
- **2. Quality Attributes are Designed In, Not Tested In:**
  - Scalability, fault isolation, `< 200ms` latency, and security must be architected from day one—they cannot be patched in via unit tests later.
- **3. Barry Boehm's Cost of Defect Removal Curve:**
  - Catching an architectural flaw in **Design** costs **$1**; in integration testing **$10**; in **production**, it escalates to **$50–$100x**!

The 4 Design Stages & Deliverables (SDD & ADRs)

> ***Chapter 5 Master Plan:*** *We teach the* ***4 Design Stages Top-Down (5.3 → 5.6)*** *for clarity, though real projects iterate freely across them!*

1. The 4 Design Stages & Flexible Entry Points

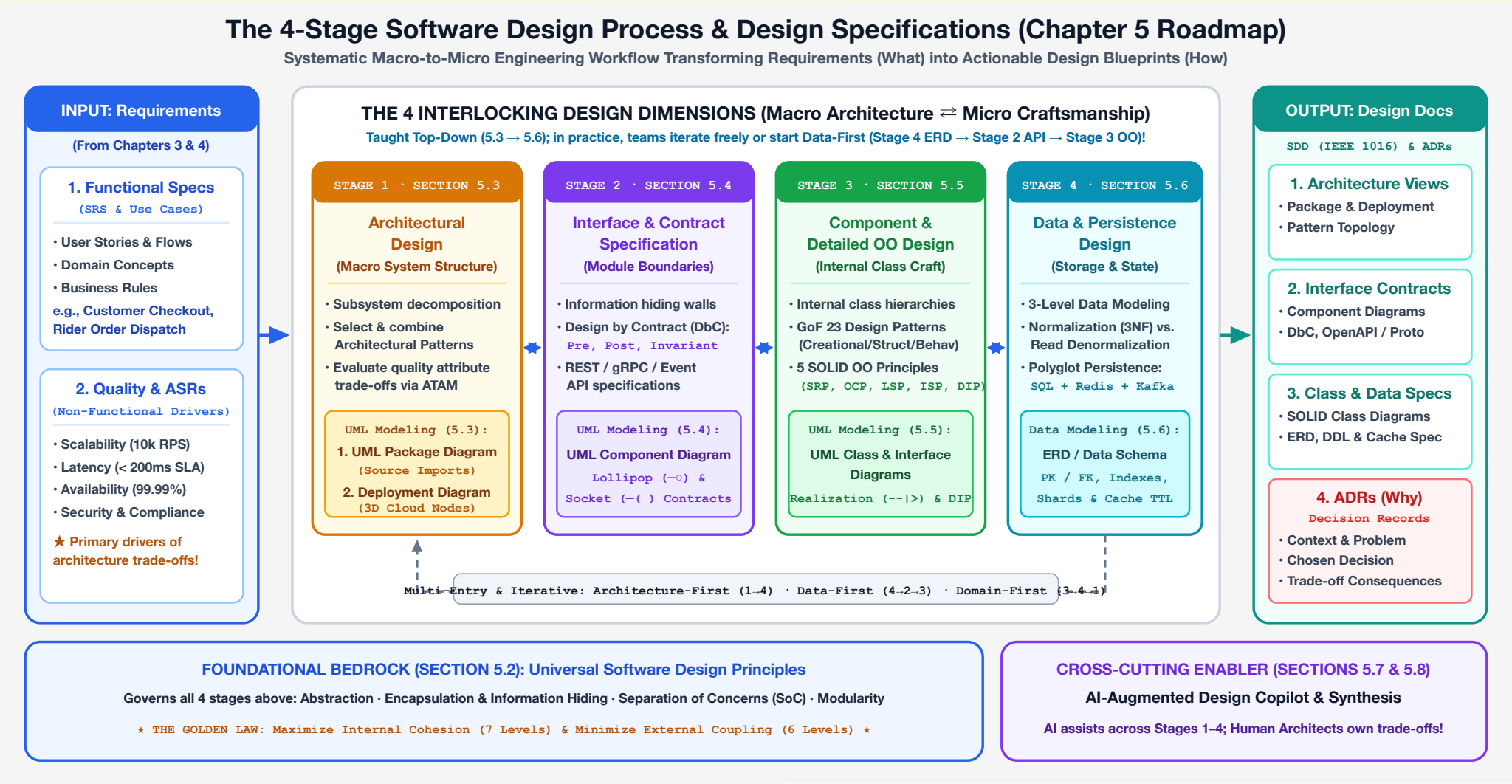
1. **Stage 1 — Architectural Design (5.3)**: Patterns, ATAM & **Package / Deployment UML**.
2. **Stage 2 — Interface & Contract Spec (5.4)**: **DbC** (`Pre/Post/Inv`) & **Component UML** (`—○`/`—(`).
3. **Stage 3 — Component & OO Design (5.5)**: GoF Patterns, **SOLID** & **Class UML**.
4. **Stage 4 — Data & Persistence Design (5.6)**: 3-Level **ERD**, 3NF, Redis & Polyglot DBs.

- **Not a Rigid Waterfall! Common Entry Points**:
  - **Architecture-First (`1 → 2 → 3 → 4`)**: High-scale distributed platforms.
  - **Data-First (`4 → 2 → 3`)**: CRUD/ERP & student projects often lock **Stage 4 ERD tables first**, then build **Stage 2 APIs** & **Stage 3 classes**!

2. Design Deliverables (SDD & ADRs)

- **Software Design Document (SDD — IEEE 1016)**:
  - **Enables Parallel Engineering**: Frontend, Backend, DBAs, and QA build and write tests simultaneously against locked specs.
  - **Contents**: Package/Deployment views, Component API contracts (OpenAPI/Protobuf), Class models & DDL schemas.
- **Architecture Decision Records (ADRs — Michael Nygard)**:
  - Lightweight, Git-versioned records (`Title`, `Status`, `Context`, `Decision`, `Consequences`) capturing **why** key trade-offs were chosen—preventing costly "*Architectural Archaeology*"!

The 4-Stage Software Design Process & Design Documents (SDD & ADR)



5.2 Universal Software Design Principles

> *"Good design is easily changeable; poor design resists change at every step."*
> *— Dave Thomas (The Pragmatic Programmer)*

Universal Principles: Fractal Across Every Scale

- **Software Design is Fractal:**
  - The same fundamental principles govern software quality regardless of zoom level:
    - **System Scale (Macro)**: Microservices partition business domains into decoupled, cohesive services.
    - **Module Scale (Meso)**: Packages and libraries isolate responsibilities behind public API boundaries.
    - **Class Scale (Micro)**: Individual classes encapsulate state and maintain a single, cohesive reason to change.
- **Core Tenets Guard Against Complexity:**
  - Computer memory is vast, but human working memory is strictly limited (Miller's Law: 7 ± 2 chunks).
  - Software principles are cognitive tools designed to keep software understandable, testable, and modifiable by human engineering teams.

Core Design Tenets: Modularity, Abstraction & SoC

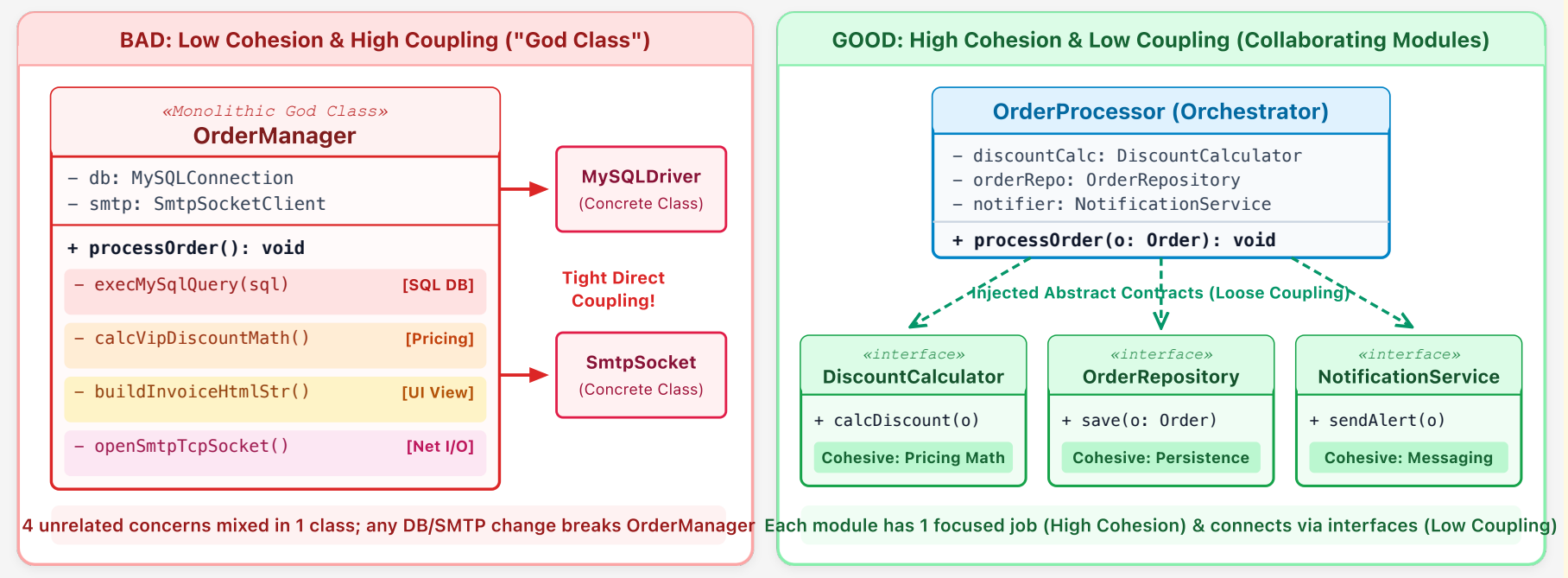
- **1. Abstraction — *[Cognitive View: "What It Does"]*:**
  - Distills complex reality into clean, high-level public contracts (`interface`) while ignoring irrelevant details, reducing developer mental load.
- **2. Encapsulation & Information Hiding — *[Protective Boundary: "How State Is Guarded"]*:**
  - *David Parnas's Principle:* Bundles state and behavior inside a capsule while locking away `private` secrets to prevent external state corruption.
- **3. Separation of Concerns (SoC) — *[Logical Dimension: "Orthogonal Aspects"]*:**
  - Separates distinct problem domains or roles (e.g., UI presentation vs. business rules vs. SQL persistence vs. logging) so concerns never tangle.
- **4. Modularity — *[Structural Packaging: "Swappable Building Blocks"]*:**
  - Physically organizes software into discrete, self-contained units (packages, libraries, microservices) that compile, test, and deploy independently.

The Golden Rule: High Cohesion & Low Coupling

- **Cohesion :**
  - Measures how tightly focused and related the internal responsibilities of a single class or module are.
  - **Engineering Goal: High Cohesion** (A module should do one thing, and do it exceptionally well).
- **Coupling :**
  - Measures the strength of direct interdependencies between separate modules.
  - **Engineering Goal: Low Coupling** (Modules communicate via abstract contracts, remaining ignorant of internal implementation details).
- **The Universal Golden Rule:**

> ***Maximize Cohesion, Minimize Coupling!***
> *High cohesion makes modules understandable and maintainable; low coupling prevents modifications from causing cascading system ripple effects.*

Cohesion & Coupling: Bad vs. Good Design (UML Comparison)



- **Bad Design (`OrderManager`)**: Cramming SQL, pricing math, HTML rendering, and SMTP sockets into one class yields **Coincidental Cohesion** and **Tight Concrete Coupling**.
- **Good Design (`OrderProcessor`)**: Delegating to focused collaborators via injected interfaces achieves **Functional Cohesion** and **Loose Abstract Coupling**.

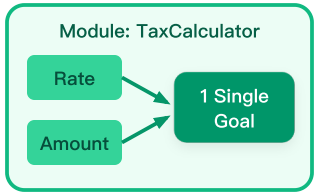
Levels and Types of Cohesion: From Weak to Strong

- **Cohesion Spectrum (Yourdon & Constantine):** How strongly focused a single module's internal elements are:

Level	Cohesion Type	Core Characteristics	Real-World Example	Quality
7	**Functional**	Performs one single, well-defined domain or math task	`computeSalesTax()`, `sin(x)`	**Ideal (Best)**
6	**Sequential**	Output of one step serves directly as input to the next	`read()` → `parse()` → `validate()`	**Strong**
5	**Communicational**	Operations share the same input or output data structure	`updateAddr()` & `printLabel()`	**Acceptable**
4	**Procedural**	Grouped by execution order, operating on different data	`checkAuth()` then `auditLog()`	**Moderate**
3	**Temporal**	Grouped purely because they execute at the same time	`initStartup()`, `closeAll()`	**Weak (Avoid)**
2	**Logical**	Similar tasks selected via an external control flag	`export(int mode)` (PDF/CSV)	**Weak (Avoid)**
1	**Coincidental**	Unrelated catch-all tasks with no meaningful relationship	`CommonUtils`, `MiscHelper`	**Worst**

Visual Spectrum: The 7 Levels of Cohesion (Strong to Weak)

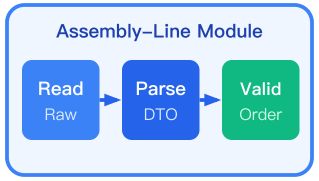
Strong & Focused Cohesion (Levels 7 → 4)



7. Functional Cohesion
(1 Single Well-Defined Task)

IDEAL (BEST)

DeliveryFeeCalc
`computeFee(distKm)`
100% Focused on 1 Math Rule

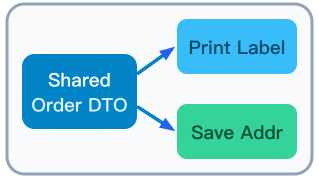


6. Sequential Cohesion
(Output of A Feeds Input of B)

STRONG

1. readOrderJson()
2. validateAndTotal()

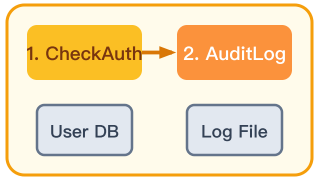
Direct Data Relay Pipeline



5. Communicational
(Operate on Same Data Record)

ACCEPTABLE

CustomerRecord
updateAddr()
printReceipt()

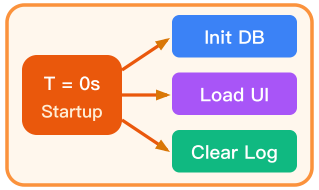


4. Procedural Cohesion
(Grouped by Order, Diff Data)

MODERATE

Step 1: checkPermission()
↓ (Control Order Only)
Step 2: writeDiskAudit()

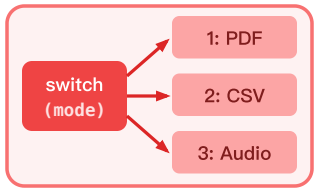
Weak & Entangled Cohesion (Levels 3 → 1)



3. Temporal Cohesion
(Same Time Slot Only)

WEAK (REFACTOR)

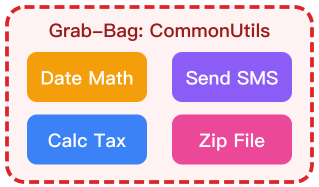
`initStartup()`
connectDb() + renderSplash()
+ pingSmtp() + loadFonts()
Unrelated Startup Batch



2. Logical Cohesion
(Flag-Controlled Switch Box)

POOR (ANTI–OCP)

`export(int flag)`
```
if (flag==1) drawPdf();
else if (flag==2) dumpCsv();
```
Tangled Multi–Branch Logic



1. Coincidental Cohesion
(Zero Meaningful Relation)

WORST (ELIMINATE)

`MiscHelper.java`
```
formatCurrency() // UI
openTcpSocket() // Net
computeVatTax() // Biz
```

Cohesion Refactoring Rule (Single Responsibility Principle)

Split Coincidental / Logical / Temporal God Classes (Levels 1–3)
→ Into Focused Functional & Sequential Collaborators (Levels 6–7)

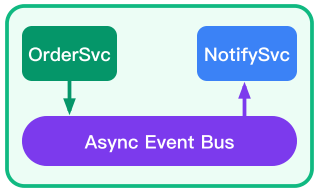
Levels and Types of Coupling: From Tight to Loose

- **Coupling Spectrum:** Measures the degree of direct interconnections and dependencies between separate modules:

Level	Coupling Type	Coupling Mechanism & Risk	Real-World Example	Quality
1	**Content (Pathological)**	Reaches directly into private fields/branches of another	Modifying a `private` field via reflection	**Worst (Severe)**
2	**Common (Global)**	Multiple modules read/write shared global mutable state	Sharing mutable global flags across threads	**High Risk**
3	**Control**	Caller dictates callee's internal execution flow via flags	Passing `mode = 3` to select internal branch	**Moderate Risk**
4	**Stamp (Data-Struct)**	Passes a whole composite object when only 1 field is needed	Passing entire `Customer` to read `zipCode`	**Acceptable**
5	**Data**	Communicates exclusively via needed primitive parameters	`calculateArea(width, height)`	**Best (Direct)**
6	**Message / Event**	Communicates asynchronously via events with zero reference	Publishing `OrderPlacedEvent` via EventBus	**Ideal (Loosest)**

Visual Spectrum: The 6 Levels of Coupling (Loose to Tight)

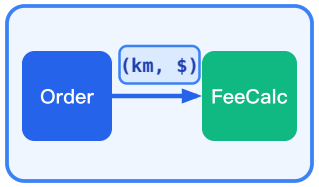
Loose & Modular Coupling (Levels 6 → 4)



6. Message / Event
(Async Events, Zero Direct Ref)

IDEAL (LOOSEST)

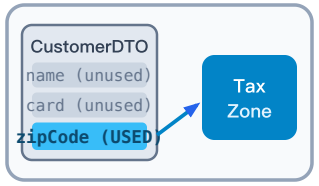
```
publish(OrderPlaced)
```
Sender & Receiver Have
0 Compile–Time Dependency



5. Data Coupling
(Passes Needed Primitives Only)

BEST (DIRECT)

```
calcFee(double km)
```
Passes Only Exact Values
Zero Knowledge of Order
or Customer Class Structure



4. Stamp Coupling
(Whole Object for 1 Field)

MODERATE

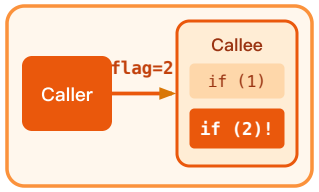
```
findZone(Customer c)
return map.get(c.zipCode);
```
Risk: Tied to Customer Class
Fix: Pass String zipCode

Loose Coupling Firewall (Levels 5–6 + Interface Contracts)

Module A → [Narrow Interface / Primitives / Event] → Module B

Result: Changing Module B Internals Causes Zero Ripple Effect on Module A

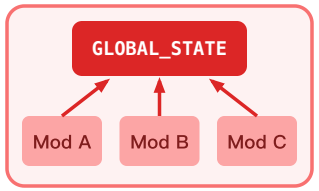
Tight & Pathological Coupling (Levels 3 → 1)



3. Control Coupling
(Flag Dictates Internal Branch)

HIGH RISK

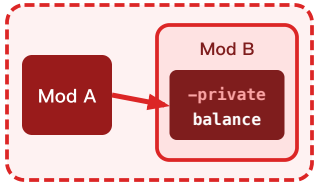
```
pay(amt, int mode)
```
Caller Must Know Internal
if/else Flag Constants
Fix: Polymorphic Strategy



2. Common (Global)
(Shared Global Mutable State)

SEVERE RISK

```
GlobalApp.currentCart
```
Hidden Side Effects &
Thread Race Conditions
Any Reader Can Corrupt All



1. Content Coupling
(Mutates Private Internals)

WORST (PATHOLOGICAL)

```
Encapsulation Breach
order.payment.rawSql
= "UPDATE paid=1";
```
Any Change in B Breaks A!

Tight Coupling Ripple Effect (Levels 1–3 Domino Failure)

Change 1 Private Field in Mod B → Breaks Mod A → Breaks Mod C

→ Refactor to Data / Event Coupling (Levels 5–6) via Narrow Interfaces

Concept Check Question 1 (CCQ 1)

In software system design, what is the primary practical benefit of combining **High Cohesion** with **Low Coupling**?

- **A.** Maximizes raw CPU execution speed by inlining all helper methods into one monolithic binary executable.
- **B.** Isolates internal class changes behind clean boundaries so modifications cause minimal ripple effects.
- **C.** Mandates storing all shared domain data structures inside a single globally mutable memory namespace.
- **D.** Eliminates the need to run automated integration test suites or continuous regression checks in CI/CD.



5.3 Stage 1 — Architectural Design: Patterns, Trade-offs & Macro UML

> *"There are no solutions, only trade-offs."*
> *— Thomas Sowell*

What is Software Architecture?

- **Formal Definition (IEEE 1471 / ISO 42010):**

> *Software architecture is the fundamental organization of a system embodied in its components, their relationships to each other and the environment, and the principles guiding its design and evolution.*

- **The Bridge Role of Architecture:**
  - **Requirements (What)**: Stakeholder needs, user stories, and constraints.
  - **Implementation (How)**: Source code, data schemas, and unit tests.
  - **Architecture (The Bridge)**: Transforms messy requirements into structured sub-systems, communication boundaries, and deployment topology.
- **Architecture in the Small vs. in the Large:**
  - **In the Small**: Internal design of individual applications (classes, modules, local interfaces).
  - **In the Large**: Enterprise-scale structures (distributed services, cloud platforms, event buses, multi-tier databases).

What is an Architectural Pattern?

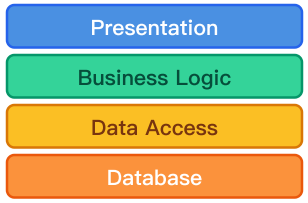
- **Definition of Architectural Patterns:**
  - An architectural pattern is a **reusable, established solution to a commonly occurring structural problem** in software system design.
  - While design patterns (e.g., Factory, Strategy) operate at the class level, architectural patterns govern the **entire macro-system topology**.
- **Three Key Elements of an Architectural Pattern:**
  - **1. Context**: The recurring design situation and engineering environment.
  - **2. Problem & Forces**: Competing requirements (e.g., decoupling UI from business logic vs. minimizing latency).
  - **3. Solution & Consequences**: Structural partitioning, communication topology, and resulting trade-offs.
- **Why Patterns Matter:**
  - Establishes a shared architectural vocabulary across engineering teams.
  - Codifies decades of engineering experience, avoiding recurring structural failures.

Architectural Patterns: Comprehensive Overview Matrix

Pattern Name	Visual Topology	Core Mechanism	Best Used For	Primary Risk & Trade-off
1. MVC	Triangular Cycle (`C → M → V`)	Separates UI presentation from data models	Interactive Web / mobile apps with rich UIs	Overkill for simple data display (CRUD)
2. Layered	Vertical Stack (`N → N-1`)	Strict hierarchy with one-way downward calls	Enterprise information systems & CRUD apps	Performance overhead; sinkhole anti-pattern
3. Repository	Hub-and-Spoke Central Store	Passive central data store with active agents	Compilers, IDEs, and shared data analysis	Central DB bottleneck & single point of failure
4. Client-Server	Distributed Network (`C ↔ S`)	Clients request services from central servers	Web apps, file servers, centralized services	Network latency & server overloading
5. Pipe-Filter	Linear Sequential Stream	Discrete transform filters connected via pipes	Data processing, compilers, ETL pipelines	Inefficient for interactive systems; parsing cost
6. Microservices	Decentralized Mesh (`S ↔ S`)	Independently deployable domain services	Large-scale, rapidly evolving cloud platforms	High operational, network & transactional complexity

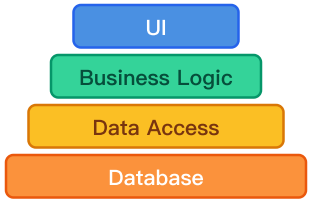
Visual Roadmap: The 6 Architectural Patterns in 5.3

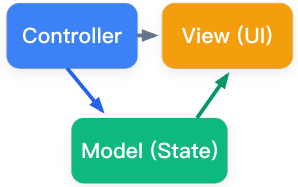
Centralized & Structural Patterns



2. Layered (N-Tier) Architecture

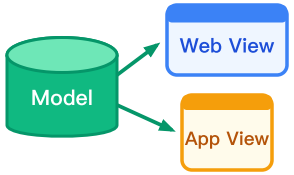
(Strict Downward Calls)

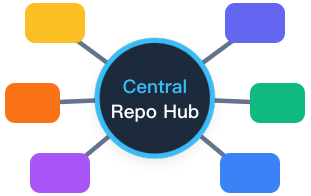




1. Model-View-Controller (MVC) Architecture

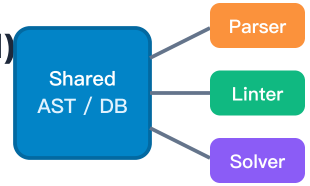
(Decoupled UI & State)

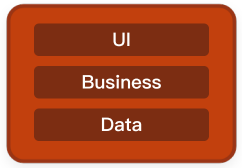




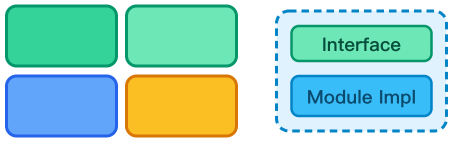
3. Repository (Blackboard) Architecture

(Central Shared Data Store)





Monolithic Architecture



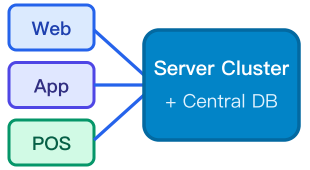
Modular Component Design

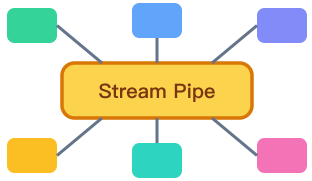
Distributed, Routing & Flow Patterns



4. Client-Server Architecture

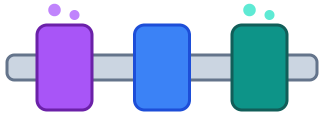
(Edge Clients ↔ Central Tier)

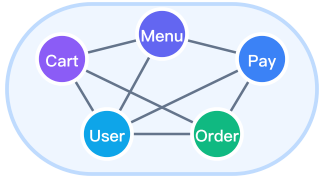




5. Pipe-and-Filter Stream Architecture

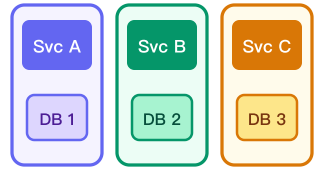
(Sequential Stateless Filters)

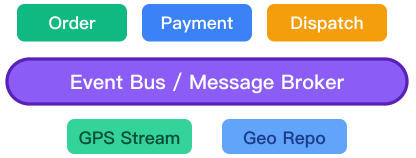




6. Microservices (MSA) Architecture

(Database-per-Service Pods)





Event–Driven Bus Topology

Real–World System Composition

Client–Server + MVC (Frontend)

+ Microservices + Layered (Services)

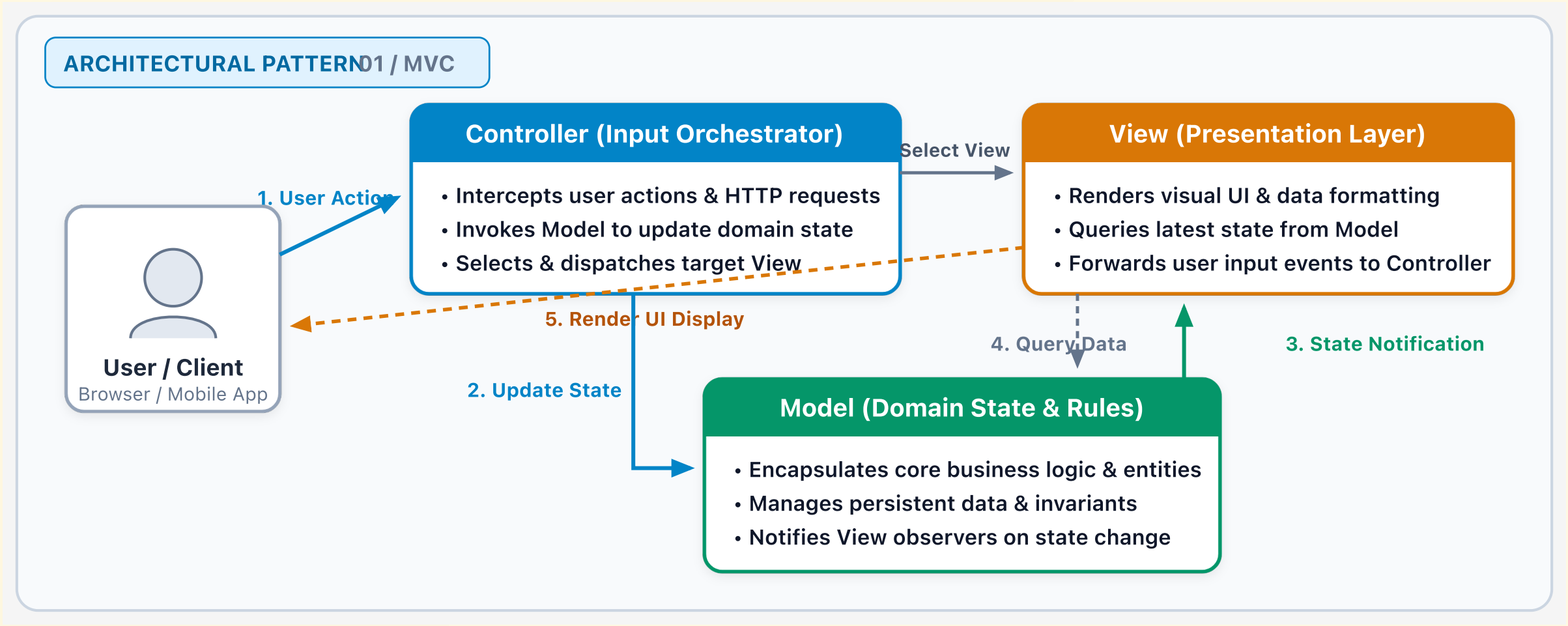
+ Pipe–Filter + Repository (Dispatch)

Multi–Pattern Hybrid Architecture

1. Model-View-Controller (MVC): Responsibilities & Trade-offs

- **Core Structural Responsibilities:**
  - **Model**: Encapsulates core business data, validation rules, and authoritative domain state; remains completely ignorant of UI presentation.
  - **View**: Renders visual representations (HTML, mobile screens, charts) to users and listens for Model state change notifications.
  - **Controller**: Intercepts user inputs and HTTP requests, orchestrates business workflows, updates the Model, and selects the target View.
- **When Used & Real-World Scenarios:**
  - Interactive Web and mobile applications where the same underlying data must be presented in multiple visual formats (e.g., dashboard tables, charts, and mobile views).
- **Engineering Advantages & Trade-offs:**
  - **Strengths**: Clean separation of concerns; UI designers and backend engineers can evolve presentation and business logic in parallel.
  - **Weaknesses**: Introduces extra indirection and boilerplate for simple CRUD screens; tight coupling arises if Views bypass Controllers to mutate state.

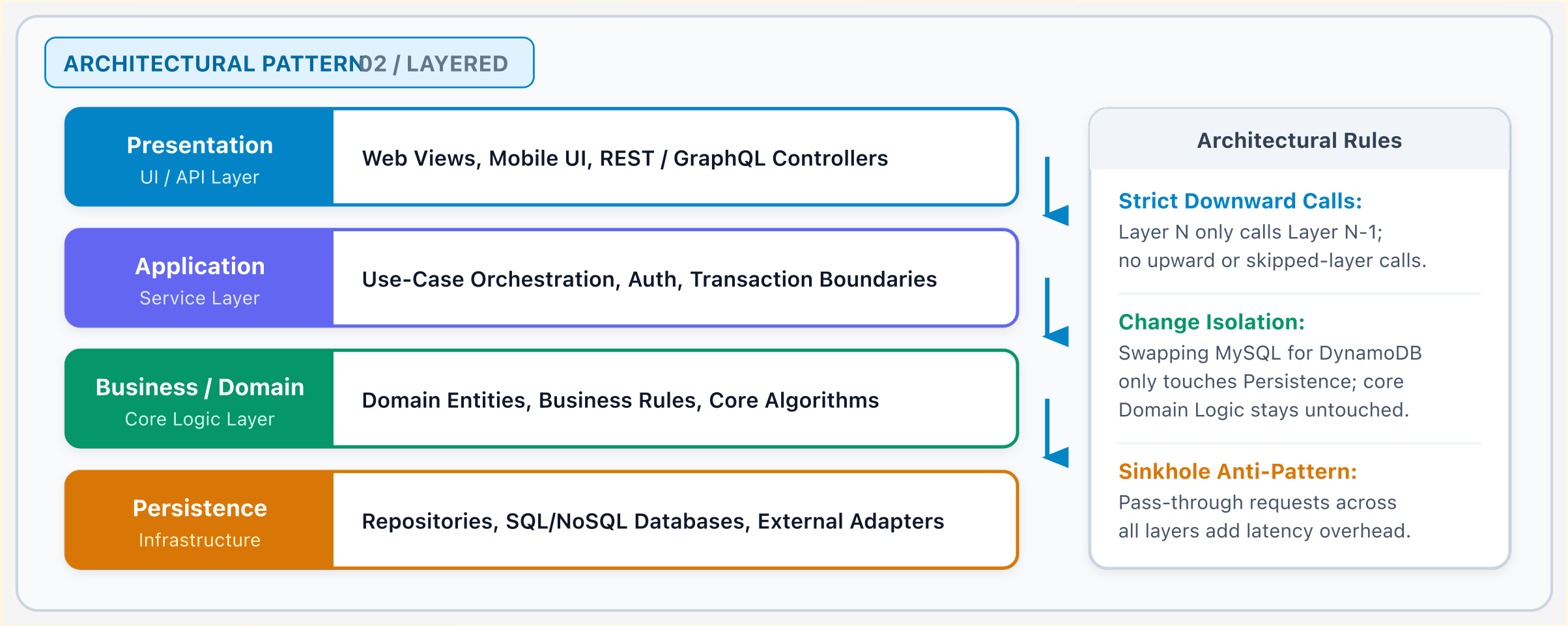
1. Model-View-Controller (MVC) Pattern — Visual Topology



2. Layered Architecture: Responsibilities & Trade-offs

- **Core Structural Responsibilities:**
  - Organizes the system into horizontal abstraction tiers: **Presentation** → **Application/Service** → **Business/Domain** → **Persistence**.
  - **Strict Layering Rule**: Each layer depends solely on the layer immediately below it (`Layer N → Layer N-1`), forbidding upward or skipped-layer calls.
- **When Used & Real-World Scenarios:**
  - Standard enterprise information systems, e-commerce backends, and banking applications requiring clear team boundaries and multi-level security isolation.
- **Engineering Advantages & Trade-offs:**
  - **Strengths**: High modularity and change isolation—swapping MySQL for DynamoDB in the Persistence layer leaves core Business Logic completely untouched.
  - **Weaknesses**: Susceptible to the **"Architecture Sinkhole Anti-Pattern"**, where requests merely pass through multiple layers without processing, adding latency and object-mapping overhead.

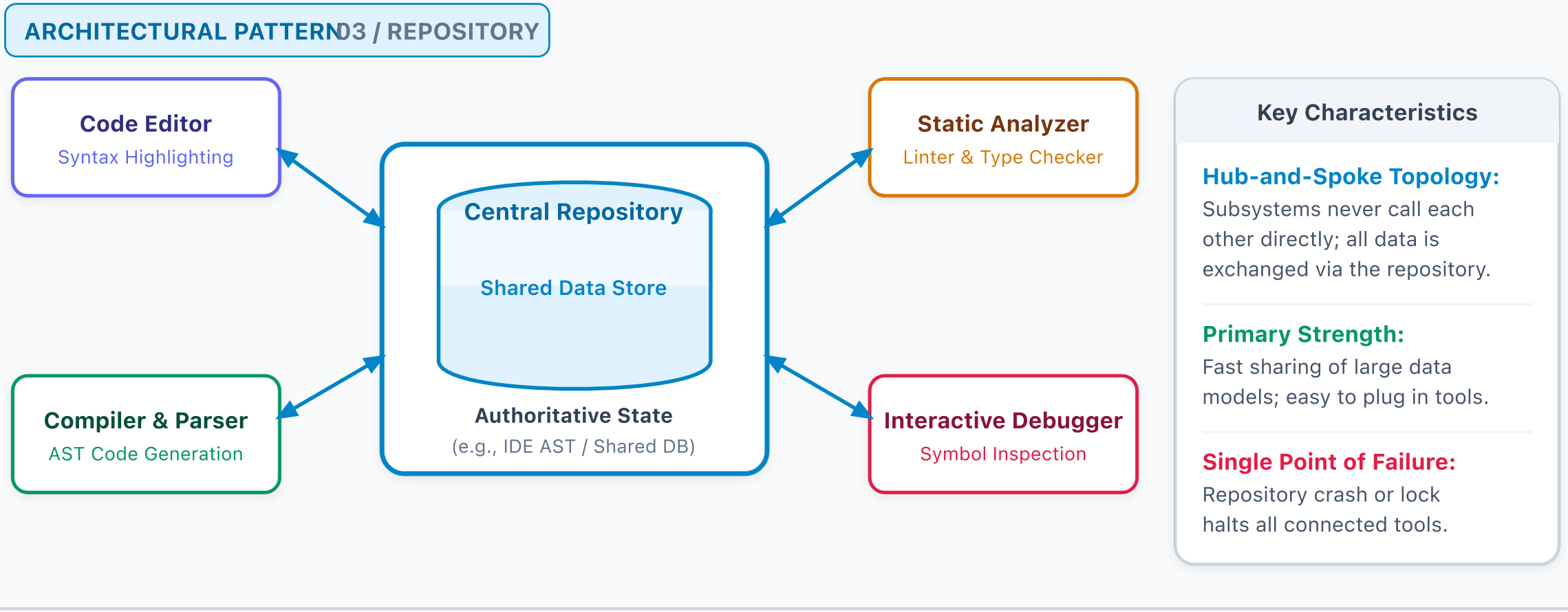
2. Layered Architecture Pattern — Visual Topology



3. Repository Architecture: Responsibilities & Trade-offs

- **Core Structural Responsibilities:**
  - **Central Repository**: A passive, authoritative data store maintaining all shared persistent state and schema invariants.
  - **Active Subsystems**: Independent tools or processing agents that never call each other directly, interacting exclusively through the central store.
- **When Used & Real-World Scenarios:**
  - Data-intensive environments where multiple specialized tools process the same complex data representation: **IDEs, compilers (shared AST), CAD systems, and AI Blackboard systems**.
- **Engineering Advantages & Trade-offs:**
  - **Strengths**: Eliminates direct inter-tool coupling; new analysis tools can be plugged into the repository effortlessly without modifying existing subsystems.
  - **Weaknesses**: The central repository is a **Single Point of Failure (SPOF)** and a major contention bottleneck under heavy concurrent writes; schema evolution affects all connected tools.

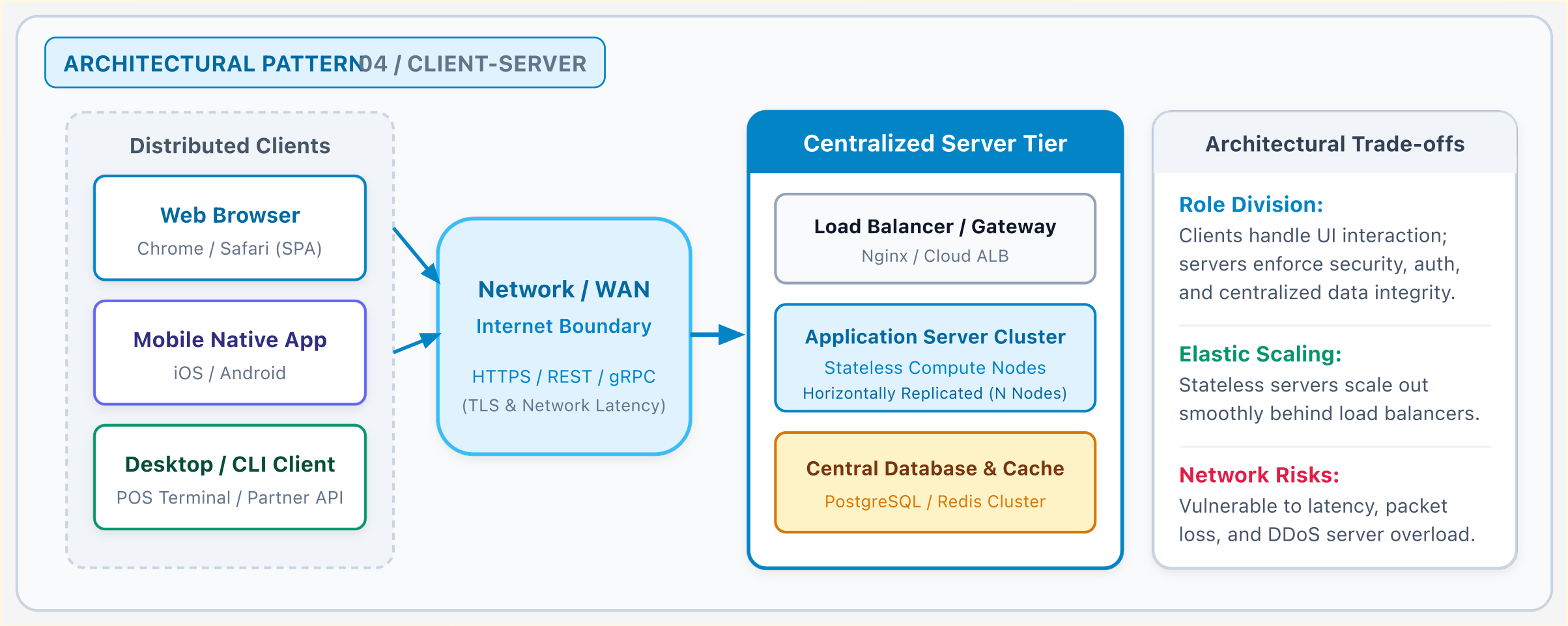
3. Repository Architecture Pattern — Visual Topology



4. Client-Server Architecture: Responsibilities & Trade-offs

- **Core Structural Responsibilities:**
  - **Client Tier**: Distributed consumer nodes responsible for user interaction, local rendering, and initiating network service requests.
  - **Server Tier**: Centralized provider nodes enforcing authentication, business rules, concurrent access control, and persistent data storage.
- **When Used & Real-World Scenarios:**
  - Web applications, mobile banking, multiplayer online services, and enterprise systems where shared data must be accessed securely from diverse locations.
- **Engineering Advantages & Trade-offs:**
  - **Strengths**: Centralizes security governance, data backups, and server-side upgrades; stateless application servers can scale horizontally behind load balancers.
  - **Weaknesses**: Vulnerable to network latency, packet loss, and connection drops; centralized server endpoints face DDoS and traffic-surge overload risks.

4. Client-Server Architecture Pattern — Visual Topology

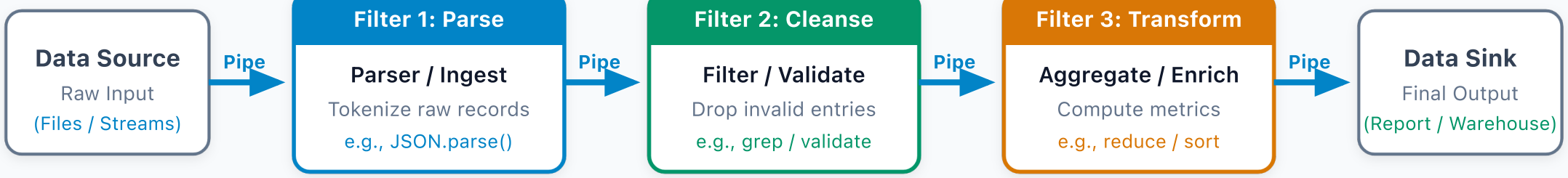


5. Pipe-and-Filter Architecture: Responsibilities & Trade-offs

- **Core Structural Responsibilities:**
  - **Filters**: Independent, stateless transformation units that consume an input stream, apply a single focused transformation, and emit an output stream.
  - **Pipes**: Unidirectional connectors that buffer and transport data streams between adjacent filters without the filters knowing each other's identity.
- **When Used & Real-World Scenarios:**
  - Multi-stage data processing pipelines: **Unix shell pipelines (`cat | grep | sort`), compiler front-ends, audio/video media encoding, and Big Data ETL workflows**.
- **Engineering Advantages & Trade-offs:**
  - **Strengths**: Exceptional composability and reuse—filters can be reordered, added, or executed concurrently across CPU cores.
  - **Weaknesses**: Ill-suited for interactive event-driven GUIs; incurs data parsing/serialization overhead at every pipe boundary and requires a common stream format.

5. Pipe-and-Filter Architecture Pattern — Visual Topology

ARCHITECTURAL PATTERN 05 / PIPE & FILTER



Terminal (Unix Shell Pipe Analogy)

```
$ cat access.log | grep "500" | sort | uniq -c
```

Each filter is stateless and decoupled from upstream/downstream stages.

High Composability: Filters can be recombined or parallelized freely.

Best Used For: ETL pipelines, compilers, stream processing

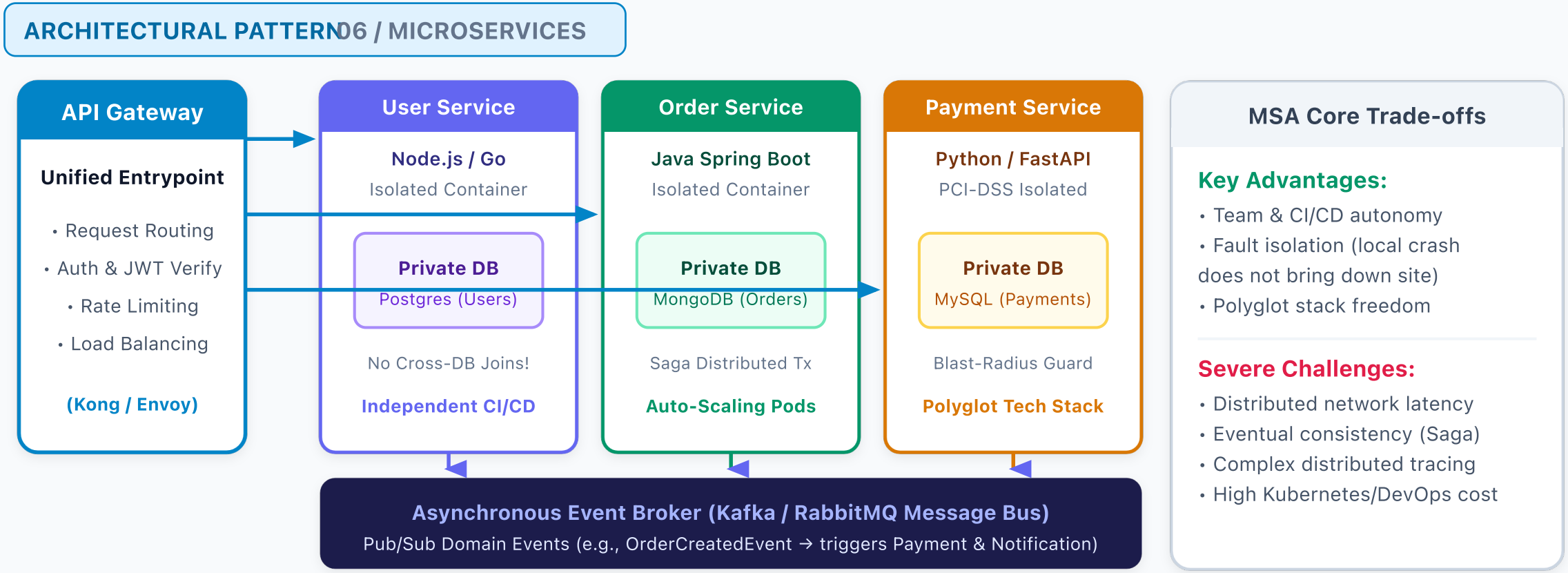
Key Strength: High reuse & easy multi-core concurrency

Trade-off: Unfit for interactive UIs; parsing overhead

6. Microservices Architecture (MSA): Responsibilities & Trade-offs

- **Core Structural Responsibilities:**
  - **Autonomous Domain Services**: Small, business-capability-aligned services running in isolated processes with independent CI/CD deployment pipelines.
  - **Database-per-Service**: Each microservice strictly encapsulates its own private database; inter-service communication occurs solely via APIs (REST/gRPC) or Event Brokers (Kafka).
- **When Used & Real-World Scenarios:**
  - Large-scale cloud platforms (e.g., Netflix, Uber, Amazon) with dozens of autonomous engineering teams requiring rapid, independent release cycles and elastic scaling per domain.
- **Engineering Advantages & Trade-offs:**
  - **Strengths**: Superior **fault isolation (blast-radius containment)**, independent scaling of bottleneck services, team autonomy, and polyglot technology freedom.
  - **Weaknesses**: High distributed systems complexity—network latency, eventual consistency (Saga transactions), distributed tracing, and heavy Kubernetes/DevOps operational cost.

6. Microservices Architecture Pattern (MSA) — Visual Topology



Monolithic vs. Microservices: Deep Dive

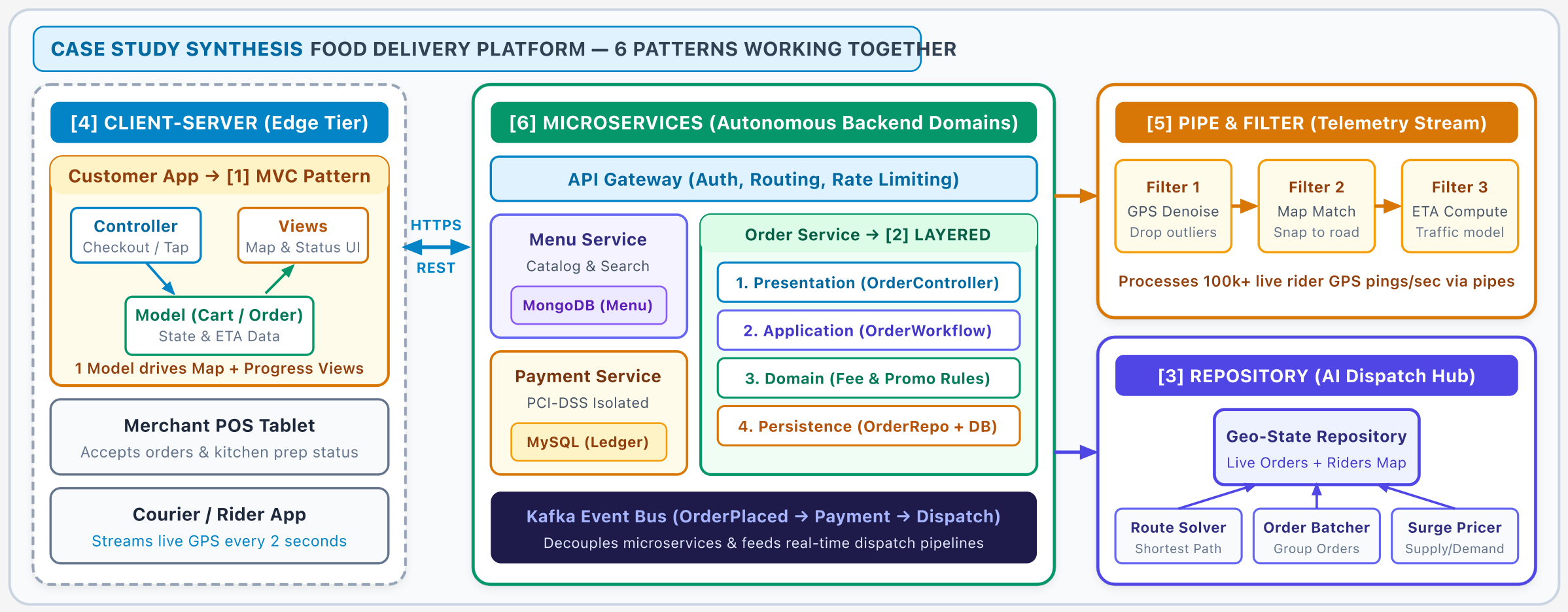
Comparison Dimension	Monolithic Architecture	Microservices Architecture
Deployment Unit	Single unified binary / WAR / container	Dozens/hundreds of independent containers
Data Storage	Single shared relational database	**Database-per-Service** (strict data encapsulation)
Communication	Fast in-memory function calls	Network RPC / REST / gRPC / Event Bus
Fault Isolation	**Poor**: Memory leak in Module A crashes entire app	**Superior**: Payment service crash leaves Browse active
Team Scalability	Merge conflicts; coordination bottlenecks	Independent feature teams deploying autonomously
Operational Cost	Simple (single CI/CD pipeline, standard logs)	High (Kubernetes, distributed tracing, service mesh)
Architectural Sweet Spot	Startups, MVP stage, small teams (<15 devs)	Large organizations (>50 devs), complex domains

Case Study Synthesis: 6 Patterns in a Food Delivery System

- **Multi-Pattern Composition:** Real systems combine patterns by **zoom level** and **problem domain**:

Pattern	Food Delivery Subsystem / Zoom Level	Core Engineering Problem Solved
1. MVC	**Customer App & Merchant Portal UI**	One `OrderModel` simultaneously updates Live Map View & Progress Bar View
2. Layered	**Inside `Order Service` (`Controller → Domain → Repo`)**	Isolates delivery fee & promo pricing rules from PostgreSQL persistence
3. Repository	**AI Dispatch Blackboard (`Geo-State Store`)**	Route solver, order batcher & surge pricer share live city map state
4. Client-Server	**Customer / Merchant / Rider Apps ↔ Cloud**	Centralizes menu pricing, auth & payment verification on secure servers
5. Pipe-Filter	**Live Rider GPS Stream (`Denoise → Match → ETA`)**	Processes 100k+ GPS pings/sec through stateless, parallelizable stages
6. Microservices	**Backend Domains (`Menu`, `Order`, `Pay`, `Dispatch`)**	Scales GPS dispatch 20x during rainstorms; isolates promo crashes from checkout

Case Study: Food Delivery System — Multi-Pattern Topology



Architecture Directly Shapes Quality Attributes

- **Quality Attributes Cannot Be Retrofitted Later:**
  - You cannot build a tightly coupled monolith and hope a few unit tests will magically make it scalable.
- **How Architectural Decisions Determine Quality Attributes:**
  - ⚡ **Performance**: Dictated by network hops, in-memory caching layers, and database partitioning.
  - 🛡️ **Security**: Dictated by zero-trust network zones, API gateways, and isolation of sensitive databases.
  - 🦺 **Safety**: Dictated by structural isolation of safety-critical functions (e.g., flight control vs. in-flight entertainment).
  - 🔄 **Availability & Fault Tolerance**: Dictated by redundancy, health checks, circuit breakers, and automated failover replicas.
  - 🔧 **Maintainability & Modifiability**: Dictated by strict adherence to High Cohesion and Low Coupling boundaries.

Architectural Trade-off Analysis & ATAM

- **The Inevitable Conflict Matrix (No Silver Bullet):**
  - **Performance vs. Security**: Deep packet inspection, TLS, and fraud checks add latency.
  - **Availability vs. Consistency (CAP Theorem)**: Strict distributed locks block orders during network partitions.
  - **Modularity vs. Performance**: Fine-grained layers and microservice RPCs add serialization overhead.
- **ATAM (Architecture Trade-off Analysis Method — Carnegie Mellon SEI):**
  - Evaluates how architectural decisions satisfy or conflict across **Quality Attribute Scenarios**:
    - **Sensitivity Points**: Decisions critical to a single quality attribute (e.g., *Circuit breakers on* `Payment Service` *directly govern checkout* ***Availability***).
    - **Trade-off Points**: Decisions where improving one quality attribute directly penalizes another.
    - **Risks & Non-Risks**: Unmitigated architectural bottlenecks vs. validated safe design choices.

ATAM Case Study: Food Delivery Architecture Trade-offs

- **Applying ATAM to 4 Critical Decisions in a Food Delivery Platform:**

Architectural Decision	Competing Quality Attributes (Trade-off Point)	ATAM Architectural Resolution
1. Rider GPS Ping Rate (`1s` vs. `15s` interval)	**Map Accuracy (Usability) ↔ Rider Battery & Bandwidth (Efficiency)**	**Adaptive Pinging**: Ping every `2s` near drop-off; throttle to `15s` when waiting at restaurant
2. Checkout & Dispatch (`Sync 2PC` vs. `Async Saga`)	**Strict Data Consistency ↔ Peak Surge Availability & Latency (CAP)**	**Hybrid Saga**: Sync pre-auth for payment, then async Kafka `OrderPlacedEvent` for dispatch
3. AI Dispatch Repository (`Central Store` vs. `Sharded`)	**Global Route Optimality ↔ Single Point of Failure Risk (SPOF)**	**Geo-Sharded In-Memory Cluster**: Partition Redis repository by city zone with active replicas
4. Promo & Fraud Check (`Deep ML` vs. `Fast Rule`)	**Anti-Fraud Security ↔ Checkout Throughput (<200ms Latency)**	**Two-Stage Pipeline**: Fast `<15ms` rule check inline at Gateway; async ML audit post-order

UML Package Diagram: Meaning, Notation & Food Delivery

> ***Architectural Meaning (Compile-Time View):*** *Models* ***source code organization, subsystem boundaries,*** *and* ***dependency direction governance.***

Core UML Notations & Rules

- **Tabbed Folder (`Package`)**: Groups cohesive classes and interfaces into a single namespace (`domain.order`).
- **Visibility (`+` Public vs. `-` Private)**: `+OrderService` & `+IOrderRepository` are exported; `-SurgePricingPolicy` is hidden inside the package.
- **Dependency Arrow (`- - ->`)**: Dashed open arrow with `«import»` or `«access»`; changes in the target package may force recompilation of the source package.
- **Acyclic Dependencies Principle (ADP)**: Package dependency graphs must never form a cycle (`A → B → A` is forbidden).

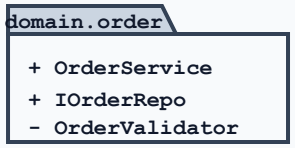
Food Delivery Design Insights

- **Bounded Context Partitioning**: Separate core domain packages for `domain.order`, `domain.dispatch`, and `domain.payment`.
- **Inward Dependency Governance**:
  - `ui.customer_app` imports `domain.order` downward.
  - `infra.persistence` (`PostgresOrderRepoImpl`) imports `domain.order` (`IOrderRepository`) **upward**!
- **Result**: The entire `domain.*` core has **zero compile-time dependencies** on PostgreSQL, Redis, Kafka, or Stripe SDKs.

Modeling Macro Structure (1): UML Package Diagram

UML Package Diagram Notation Legend

1. Package Symbol (Tabbed Folder)



Groups logically cohesive classes, interfaces & sub-packages into a single namespace boundary.

Name in top tab or body center.

2. Element Visibility Specifiers

`+ Public Element`
Accessible by external packages

`- Private Element`
Hidden internal implementation

3. Dependency Arrow & Stereotypes



- Dashed open arrow: changes in Supplier may break Client.
- *«import»*: adds supplier public names to client namespace.
- *«access»*: private import (does not re-export elements).

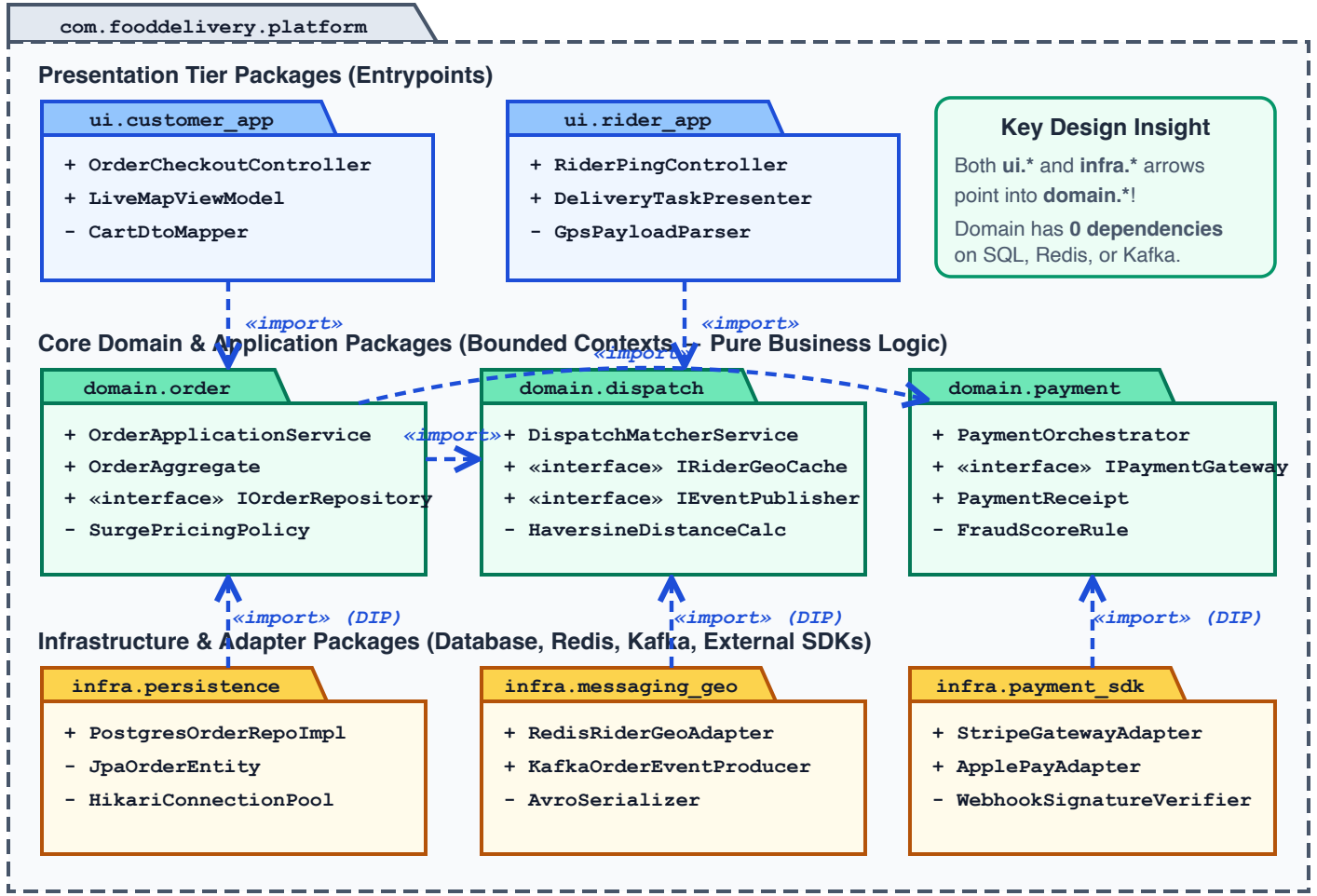
4. Acyclic Dependencies Principle (ADP)

- Package dependency graph must be a **Directed Acyclic Graph**.
- **UI & Infrastructure** depend on **Domain** (never reverse!).



No Cycles!

Food Delivery System — UML Package Diagram (Module & Dependency Structure)



Peer Discussion: Chess Game in MVC — UML Package Diagram (PlantUML)

Hands-On Modeling: Architecting a Chess Game in MVC (4 Mins)

- **Scenario:** Your team is designing a desktop/web **Chess Game** using the **Model-View-Controller (MVC)** architectural pattern.
- **1. Partition Classes into 3 Cohesive Packages (`package` syntax):**
  - `chess.model`: `+Board`, `+Piece`, `+MoveValidator`, `-BitboardEngine` (pure chess rules & state).
  - `chess.view`: `+BoardCanvasUI`, `+MoveHistoryPanel`, `+SoundEffectPlayer` (UI rendering).
  - `chess.controller`: `+GameController`, `+TurnClockHandler` (user input routing).
- **2. Draw the Package Diagram in PlantUML (PlantText):**
  - Use `package "chess.model" { ... }` and connect packages via `..> : «import»`.
  - **Audit Question:** Which direction do the dependency arrows point? Does `chess.model` import `chess.view`? Does your diagram satisfy the **Acyclic Dependencies Principle (ADP)**?



UML Deployment Diagram: Meaning, Notation & Food Delivery

> ***Architectural Meaning (Runtime Infrastructure View):*** *Models* ***physical hardware nodes, cloud runtime containers, deployable binary artifacts, and network protocols.***

Core UML Notations & Rules

- **3D Node Cube (`«device»` / `«executionEnv»`)**:
  - `«device»`: Physical compute host (Mobile phone, AWS EC2, Load Balancer).
  - `«executionEnvironment»`: Nested runtime container (Kubernetes Pod, Docker, JVM, PostgreSQL Engine).
- **Artifact (`«artifact»` + 📄 Icon)**: Concrete deployable binary/file (`order-service:v2.4.jar`, `rider-gps.apk`) that `«manifest»` a logical component.
- **Communication Path & Multiplicity**: Solid line labeled with network protocol (`«gRPC / HTTP2»`, `«JDBC / TLS»`) and node cardinality (`[5..20]`).

Food Delivery Design Insights

- **Exposes High-Availability & Scaling Policies**:
  - `Dispatch Engine Pod [5..20]` scales wider than `Payment Pod [2..6]` to handle 6 PM dinner GPS spikes.
- **Eliminates Single Points of Failure (SPOF)**:
  - Shows `RDS PostgreSQL [1 Primary + 2 Replica]`, `Redis ElastiCache [6 Shards]`, and `Kafka [3 Nodes]`.
- **Makes Network Boundaries Explicit**:
  - External traffic uses `«HTTPS / REST & WSS»`, internal east-west traffic uses `«gRPC»`, and Payment uses `«HTTPS / mTLS»` to `Stripe Cloud`.

Modeling Macro Topology (2): UML Deployment Diagram

UML Deployment Diagram Notation Legend

1. Node (3D Cube: Hardware or Runtime Env)

«device»
AWS EC2 Host
«executionEnv» Docker

- **«device»**: Physical compute resource (Server, Phone, IoT).
- **«executionEnvironment»**: OS, Docker Container, K8s Pod, JVM, or DBMS nested inside.

2. Deployable Artifact («artifact»)

«artifact»
order-svc.jar

Concrete physical file deployed onto a Node (e.g., `.jar`, `.war`, `.ipa`, Docker image, SQL schema) that «manifest» a component.

3. Communication Path & Multiplicity

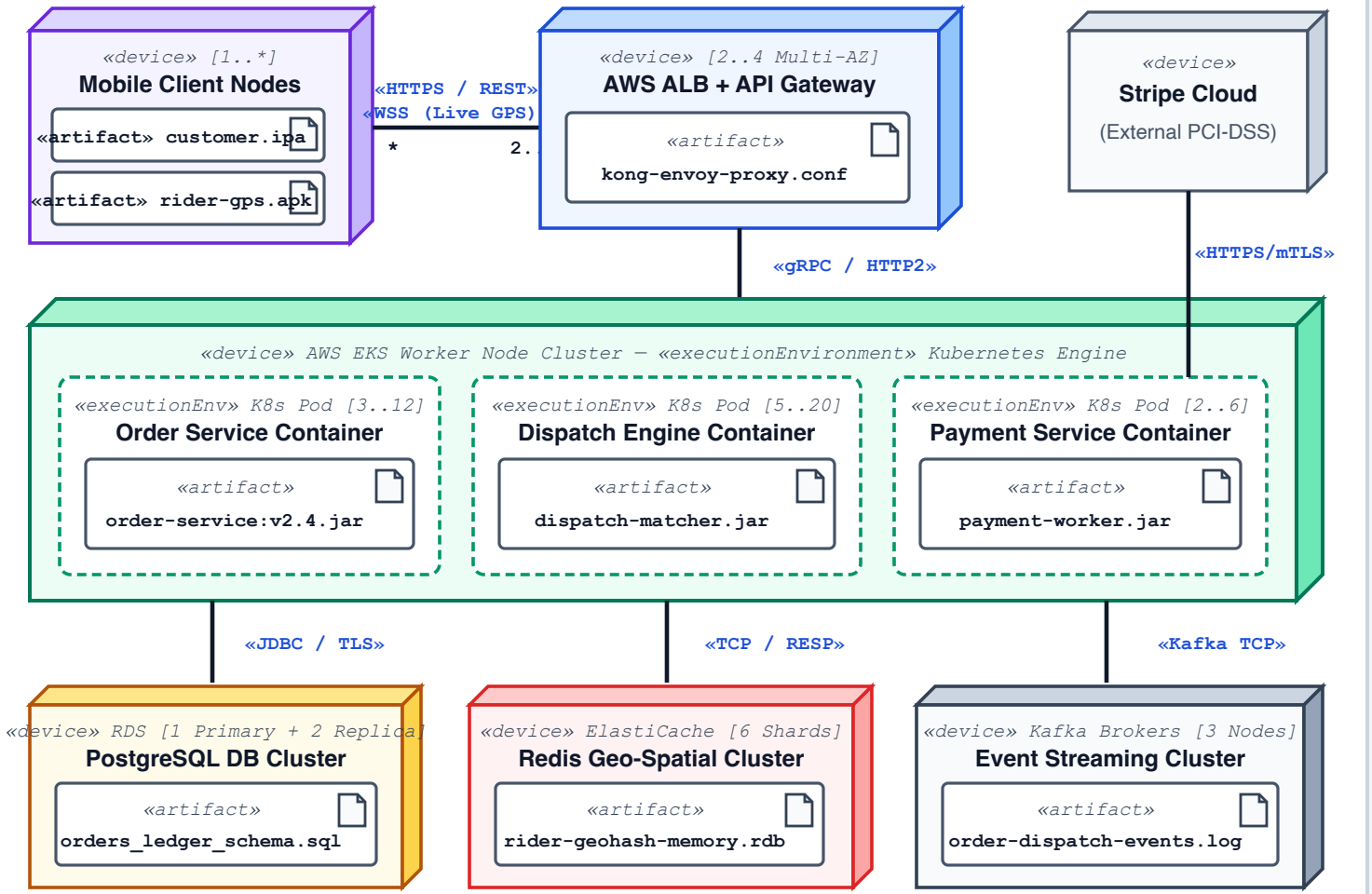
Node A — «gRPC / TLS 1.3» — Node B
1 — 3..1

- Solid line = network link + transport protocol & cardinality.

4. Why Deployment Diagrams Are Vital

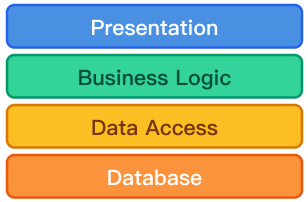
- Bridges **software binaries** to **cloud infrastructure**.
- Reveals **Single Points of Failure (SPOF)**, firewall boundaries, replication topology & auto-scaling pools that are 100% invisible in Class or Package diagrams!

Food Delivery System — UML Deployment Diagram (Cloud Infrastructure)



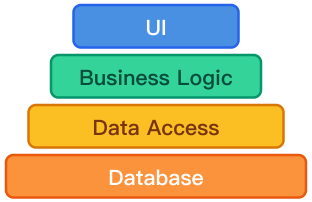
Section 5.3 Visual Recap: Structural vs. Distributed Patterns

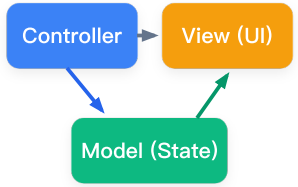
Centralized & Structural Patterns



2. Layered (N-Tier) Architecture

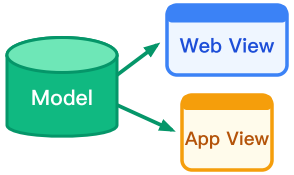
(Strict Downward Calls)

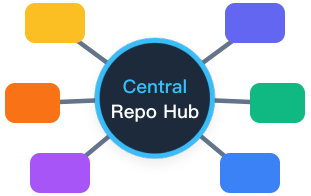




1. Model-View-Controller (MVC) Architecture

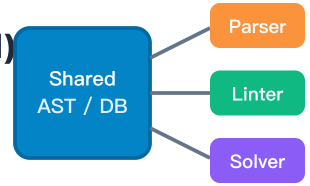
(Decoupled UI & State)

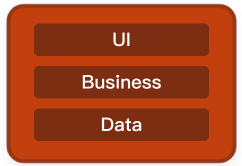




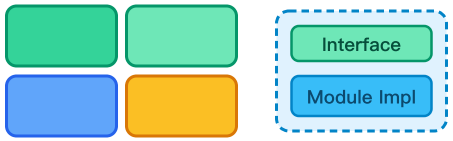
3. Repository (Blackboard) Architecture

(Central Shared Data Store)





Monolithic Architecture



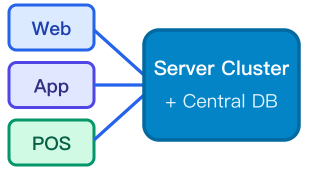
Modular Component Design

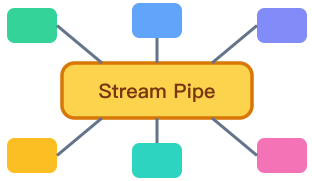
Distributed, Routing & Flow Patterns



4. Client-Server Architecture

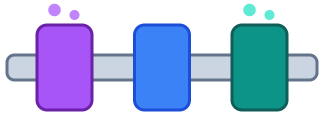
(Edge Clients ↔ Central Tier)

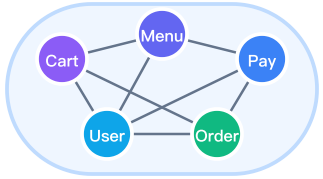




5. Pipe-and-Filter Stream Architecture

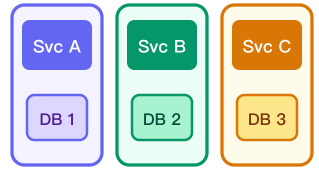
(Sequential Stateless Filters)

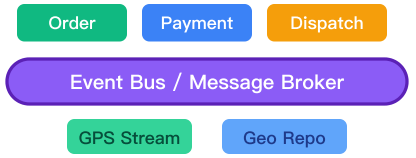




6. Microservices (MSA) Architecture

(Database-per-Service Pods)





Event–Driven Bus Topology

Real–World System Composition

Client–Server + MVC (Frontend)

+ Microservices + Layered (Services)

+ Pipe–Filter + Repository (Dispatch)

Multi–Pattern Hybrid Architecture

Concept Check Question 2 (CCQ 2)

When comparing a Monolithic architecture with a Microservices architecture, what is the primary structural reason that Microservices provide superior **fault isolation** (blast radius containment)?

- **A.** All services read and write to a single shared enterprise SQL database instance with global table locks.
- **B.** Each service runs in an isolated process with a private database, containing the blast radius of crashes.
- **C.** Distributed service calls completely eliminate network serialization overhead and inter-node TCP latency.
- **D.** Decomposing a monolith into microservices drastically reduces DevOps infrastructure and monitoring costs.



5.4 Stage 2 — Interface & Contract Specification: DbC & Component UML

> *"Treat every software module boundary as a formal legal contract with explicit rights and obligations."*
> *— Bertrand Meyer (Creator of Design by Contract)*

Design by Contract (DbC): Rights, Obligations & API Specs

> ***Core Principle (Bertrand Meyer):*** *If the* ***Client*** *fulfills the* ***Precondition****, the* ***Supplier*** *is contractually bound to deliver the* ***Postcondition*** *while preserving the* ***Class Invariant****.*

The 3 Pillars of Design by Contract (DbC)

- **1. Precondition (`requires` — Client Obligation)**:
  - What must be true *before* invocation (`!cart.isEmpty() && token.isValid()`).
  - *Benefit to Supplier*: No need to litter internal code with redundant defensive checks; if violated, fail fast!
- **2. Postcondition (`ensures` — Supplier Guarantee)**:
  - What the Supplier guarantees *after* execution (`order.status == CONFIRMED && stock_after == stock_before - qty`).
  - *Benefit to Client*: Guaranteed outcome without inspecting Supplier internals.
- **3. Class Invariant (`invariant` — Domain Integrity)**:
  - Always true before and after any operation (`order.totalAmount >= 0`).

Industrial Interface Specification Standards

- **Eliminating Ambiguity Across Teams**:
  - Frontend, Mobile, and Backend teams lock interface specs in the **SDD** on Day 1 to enable **parallel development & contract testing**.
- **Modern Machine-Readable Contract Formats**:
  - **REST APIs**: **OpenAPI 3.1 (Swagger)** — specifies endpoints, JSON schemas, status codes (`201`, `400`, `422`), and validation bounds.
  - **RPC Services**: **Protocol Buffers (`.proto`)** — strongly-typed binary contracts for east-west `gRPC` calls.
  - **Event Streams**: **AsyncAPI / Apache Avro** — versioned schemas for Kafka `OrderPlacedEvent` payloads + **Idempotency Keys** (`idempotency_key: UUID`) to prevent duplicate payment charging!

Formal Interface Specification & Design by Contract (DbC)

How to Specify an Interface Contract: Design by Contract (DbC)

Example: Food Delivery Checkout Interface — IOrderCheckoutAPI.placeOrder(cart, paymentToken) : OrderReceipt

CLIENT (Caller)

Customer App / API Gateway

Invokes operation:

```
placeOrder(cart, token)
```

Must satisfy Precondition before calling!

1. PRECONDITION (requires)

Client's Obligation Before Call

- `cart != null`
- `cart.items.size() >= 1`
- `paymentToken.isValid()`
- `restaurant.isOpenNow()`

If violated → Caller is at fault!

SUPPLIER (Callee)

«component» OrderService

Executes business logic (Internal implementation hidden behind interface)

Assumes Precondition holds; delivers Postcondition!

2. POSTCONDITION (ensures)

Supplier's Guarantee on Return

- `receipt.status == CONFIRMED`
- `stock_after == stock_pre - qty`
- `payment.isCaptured() == true`
- `OrderPlacedEvent emitted`

If violated → Supplier has a bug!

3. CLASS INVARIANT (invariant — Domain Integrity Rules That Must Hold Before AND After Every Method Call)

Financial Integrity Invariant

`order.totalAmount >= 0`

(Order total can never be negative)

Order Structural Invariant

`sum(items.price) + fee == total`

(Line items always reconcile to total)

Dispatch Capacity Invariant

`rider.activeOrders <= 3`

(A courier never holds > 3 active orders)

Specifying `OrderService` in Code: Javadoc vs. JML

> ***From Diagram to Executable Spec:*** *How do we write* `OrderService.placeOrder()` *contracts in Java using* ***Contract Javadoc*** *and* ***JML (Java Modeling Language)****?*

1. Contract-Enhanced Javadoc Specification

```
public interface OrderService {
  /**
   * Places and confirms a food delivery order.
   *
   * @invariant activeOrders >= 0
   * @param cart  customer cart (non-null, >= 1 item)
   * @param token authorized payment token
   * @return confirmed OrderReceipt with order ID
   * @throws IllegalArgumentException if inputs invalid
   * @pre  cart != null && !cart.isEmpty()
   *       && token != null && token.isValid()
   *       && stock >= cart.totalQty()
   * @post RESULT != null
   *       && RESULT.getStatus() == Status.CONFIRMED
   *       && stock == OLD(stock) - cart.totalQty()
   */
  OrderReceipt placeOrder(Cart cart, PaymentToken token);
}
```

2. Formal JML (Java Modeling Language) Spec

```
public class OrderService {
  //@ spec_public
  private int stock;
  //@ public invariant stock >= 0;

  /*@ public normal_behavior
    @   requires cart != null && !cart.isEmpty();
    @   requires token != null && token.isValid();
    @   requires stock >= cart.totalQty();
    @   assignable stock;
    @   ensures \result != null;
    @   ensures \result.status == Status.CONFIRMED;
    @   ensures stock == \old(stock) - cart.totalQty();
    @ also
    @ public exceptional_behavior
    @   requires cart == null || !token.isValid();
    @   assignable \nothing;
    @   signals_only IllegalArgumentException;
    @*/
  public OrderReceipt placeOrder(Cart cart, PaymentToken token);
}
```

Peer Discussion: Write the Javadoc Design Spec for `bubbleSort(int[] arr)`

Pair Challenge: Complete the Javadoc Design Specification for Bubble Sort (4 Mins)

- **Task:** Write a complete, mathematically airtight **Javadoc Design Spec** (`@param`, `@pre`, `@post`, `@throws`) for `bubbleSort`:

```
/**
 * Sorts the specified array into ascending numerical order in-place.
 * @param  arr the integer array to be sorted in-place
 * @throws IllegalArgumentException if _____________________________
 * @pre    _____________________________________________________
 * @post   1. (Ordered)     forall i in [0, arr.length-2]: arr[i] <= arr[i+1]
 * @post   2. (Permutation) _____________________________________
 */
public static void bubbleSort(int[] arr)
```



- **Critique with Partner:** Why is `@post 1 (Ordered)` alone broken if a buggy implementation runs `Arrays.fill(arr, 0)`? How do you write `@post 2 (Permutation)` and frame conditions so no element is lost or fabricated?

UML Component Diagram: Meaning, Notation & Food Delivery

> ***Stage 2 Modeling Purpose:*** *Models* ***modular, replaceable software components*** *wired strictly through* ***plug-and-play interface contracts*** *(visualizing* ***Design by Contract****).*

Core UML Notations & DbC Mapping

- **Component (`«component»` + ⧉ Icon)**: Encapsulated, replaceable unit whose internal classes are hidden behind explicit interfaces.
- **Provided Interface (`—○` Lollipop / Ball)**: Contract realized by a Supplier component (`IOrderCheckoutAPI` — guarantees DbC **Postconditions**).
- **Required Interface (`—(` Socket)**: Contract demanded by a Client component (`IPaymentGateway` — must satisfy DbC **Preconditions**).
- **Assembly Connector (`—(○—` Ball & Socket)**: Mates a required socket to a provided lollipop with zero implementation coupling.
- **Port (`□`) & Delegation (`- - ->`)**: Boundary endpoint forwarding external calls to internal sub-components.

Food Delivery Design Insights

- **Composite Subsystem Decomposition**: `Order Management` encapsulates internal `OrderStateEngine` and `MenuPricingCalc` behind an external boundary Port (`□`).
- **Hot-Swappable Plug-and-Play**:
  - `Order Management` only binds to socket `IPaymentGateway`.
  - Whether the lollipop is provided by `Stripe`, `ApplePay`, or an in-memory `MockPaymentStub` during CI testing, `Order Management` requires **zero code changes**!
- **Geo-Dispatch Contract**: `Rider Dispatch Engine` queries `Rider Geo-Location Tracker` strictly via `IGeoSpatialIndex`.

Modeling Interface Contracts: UML Component Diagram

UML Component Diagram Notation Legend

1. Component Classifier («component»)

«component»
OrderService

A modular, replaceable unit that encapsulates state & behavior behind explicit interfaces.
Icon: 2-tabbed box in top-right.

2. Provided Interface — "Lollipop / Ball" (—○)

Provider —○ IPaymentAPI

Contract that the component **realizes & exposes** to clients (e.g., REST/gRPC/Service API).

3. Required Interface — "Socket" (—( )

Consumer —(IPaymentAPI

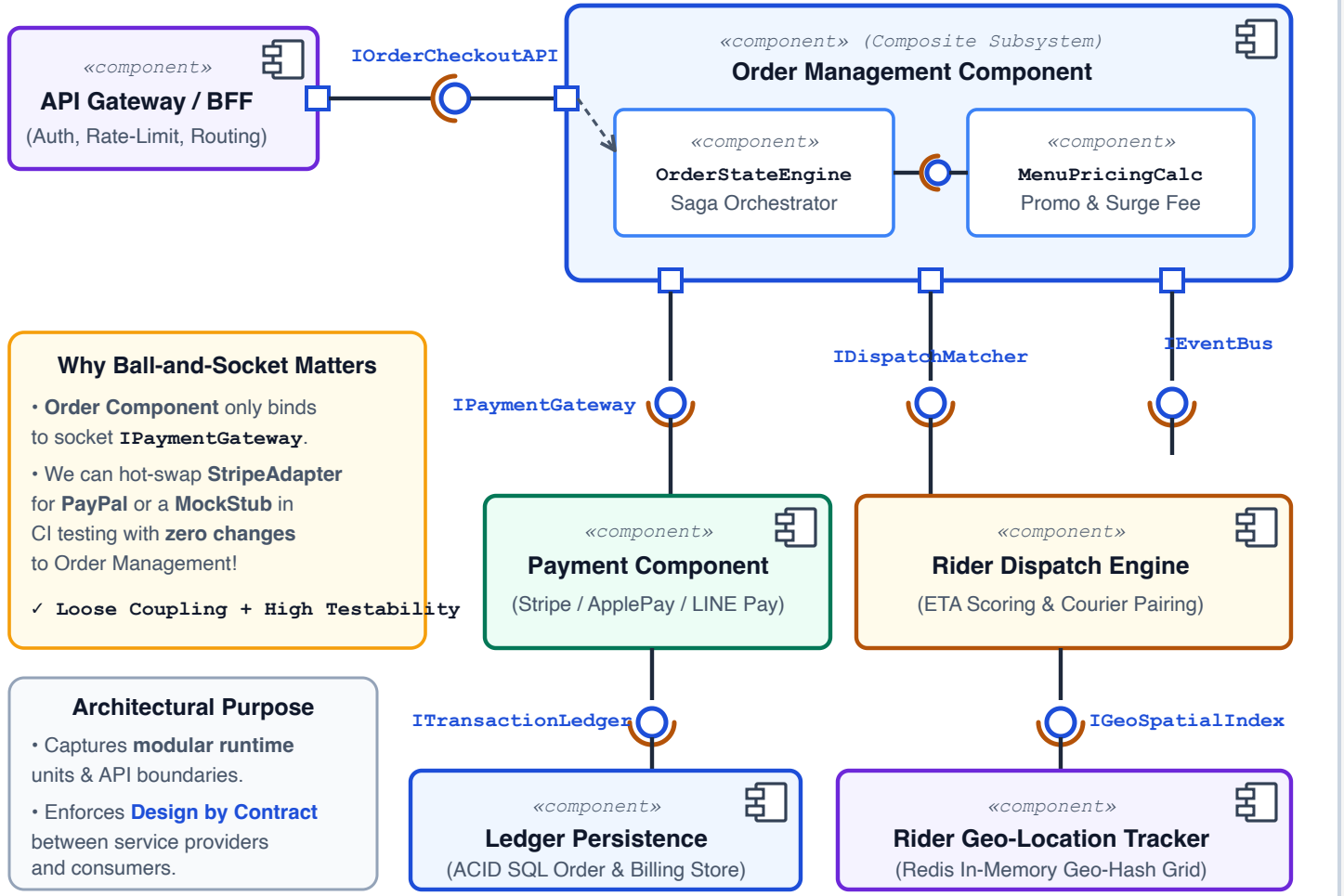
Contract that the component **requires / depends on** from an external provider to operate.

4. Assembly Connector (Ball-and-Socket) & Port (□)

Order —○ IPayment —(StripePay Plug & Play!

- **Assembly Connector —(○—**: Wires consumer socket to provider lollipop. Consumer never knows provider's concrete class!
- **Port (□ on edge)**: Explicit boundary endpoint isolating external traffic & delegating (`-->`) to internal parts.

Food Delivery System — UML Component Diagram (Interface Contracts)



Concept Check Question 3 (CCQ 3)

In Stage 2 (**Interface & Contract Specification**), how do Bertrand Meyer's **Design by Contract (DbC)** and the **UML Component Diagram** work together when `Order Management` calls `Payment Component` via the `IPaymentGateway` interface?

- **A.** `Order Management` exposes a Lollipop (`—○`) and may freely pass negative order amounts without validation.
- **B.** Client Socket (`—(`) must satisfy the **Precondition**, while Supplier Lollipop (`—○`) guarantees the **Postcondition**.
- **C.** Component diagrams use 3D hardware node cubes (`«device»`) to bypass runtime Precondition and Postcondition checks.
- **D.** Design by Contract requires the caller component to directly mutate private SQL tables inside the supplier module.

5.5 Stage 3 — Component & Detailed OO Design: Patterns, SOLID & Class UML

> *"Clean code always looks like it was written by someone who cares."*
> *— Robert C. Martin (Uncle Bob)*

What Are Design Patterns? From GoF to SOLID Foundations

- **What Is a Software Design Pattern?**
  - While **Architectural Patterns** (Section 5.3) define system-wide macro topology, **Design Patterns** provide reusable, proven micro-level solutions to recurring class and object design problems within a specific context.
- **The Vast Landscape of Design Patterns (e.g., Gang of Four — GoF 23 Patterns):**
  - **Creational Patterns (5)**: *Factory Method, Abstract Factory, Builder, Singleton, Prototype* (decouple object creation).
  - **Structural Patterns (7)**: *Adapter, Decorator, Facade, Proxy, Composite, Bridge* (compose classes & interfaces).
  - **Behavioral Patterns (11)**: *Strategy, Observer, Command, State, Template Method, Iterator* (manage object collaboration).
- **Our Focus Here — The 5 SOLID Design Principles (Robert C. Martin):**
  - Rather than memorizing 23 GoF patterns by rote, we focus on **SOLID**—the **foundational "first principles"** upon which nearly all GoF patterns are built (e.g., *Strategy* realizes **OCP**; *Adapter* realizes **DIP**)!

Overview: The 5 SOLID Design Principles in UML

- **How SOLID Shapes Class & Interface Topologies:**
  - **S — Single Responsibility Principle (SRP)**: Partition bloated "God Classes" so each class serves **1 Actor / 1 Reason to Change**.
  - **O — Open / Closed Principle (OCP)**: Replace rigid concrete dependencies with **polymorphic** `«interface»` **abstractions** (`- - -|>`).
  - **L — Liskov Substitution Principle (LSP)**: Ensure every subclass (`——|>`) or realization (`- - -|>`) honors the **behavioral contract** of its base type.
  - **I — Interface Segregation Principle (ISP)**: Split bloated "Fat Interfaces" into **lean, role-specific** `«interface»` **contracts**.
  - **D — Dependency Inversion Principle (DIP)**: Invert dependency arrows **upward** so both high-level policies and low-level details depend on abstractions.

S — Single Responsibility Principle (SRP): Design Analysis

> ***Definition (Robert C. Martin):*** *"A class should have one, and only one, reason to change (i.e., be responsible to one, and only one, actor/stakeholder)."*

❌ UML Anti-Pattern: The "God Class"

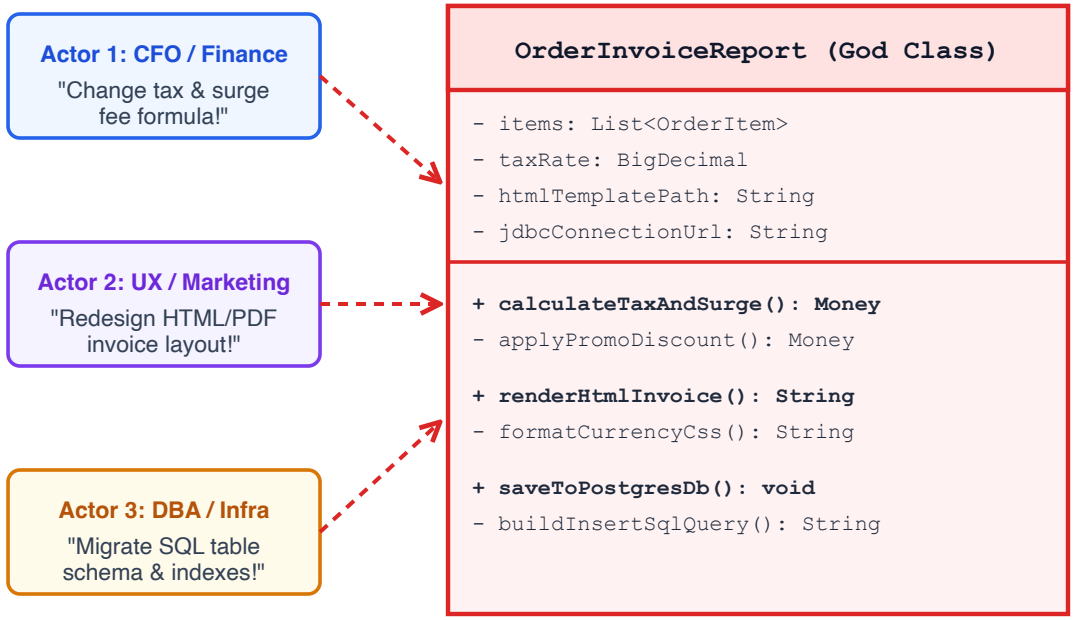
- **Tangled Axes of Change**: `OrderInvoiceReport` combines **Financial Rules** (`calculateTaxAndSurge`), **Presentation Formatting** (`renderHtmlInvoice`), and **Database I/O** (`saveToPostgresDb`).
- **Fragility & Merge Collisions**:
  - When UX designers tweak receipt CSS inside `OrderInvoiceReport`, they risk breaking CFO tax logic or triggering SQL regressions.
  - achieves low **Coincidental / Logical Cohesion**.

✅ UML Refactoring: 1 Class per Actor

- **Cohesive Decomposition**:
  - `InvoiceTaxCalculator` ➔ Serves **CFO / Finance**.
  - `InvoiceHtmlRenderer` ➔ Serves **UX / Marketing**.
  - `InvoiceSqlRepository` ➔ Serves **DBA / Infra**.
- **Architectural Payoff**:
  - Achieves **Level 1 Functional Cohesion** and **Data Coupling** (passing only `OrderInvoice`).
  - Each class can be modified, tested, and deployed independently with zero cross-team interference.

S — Single Responsibility Principle (SRP): UML Comparison

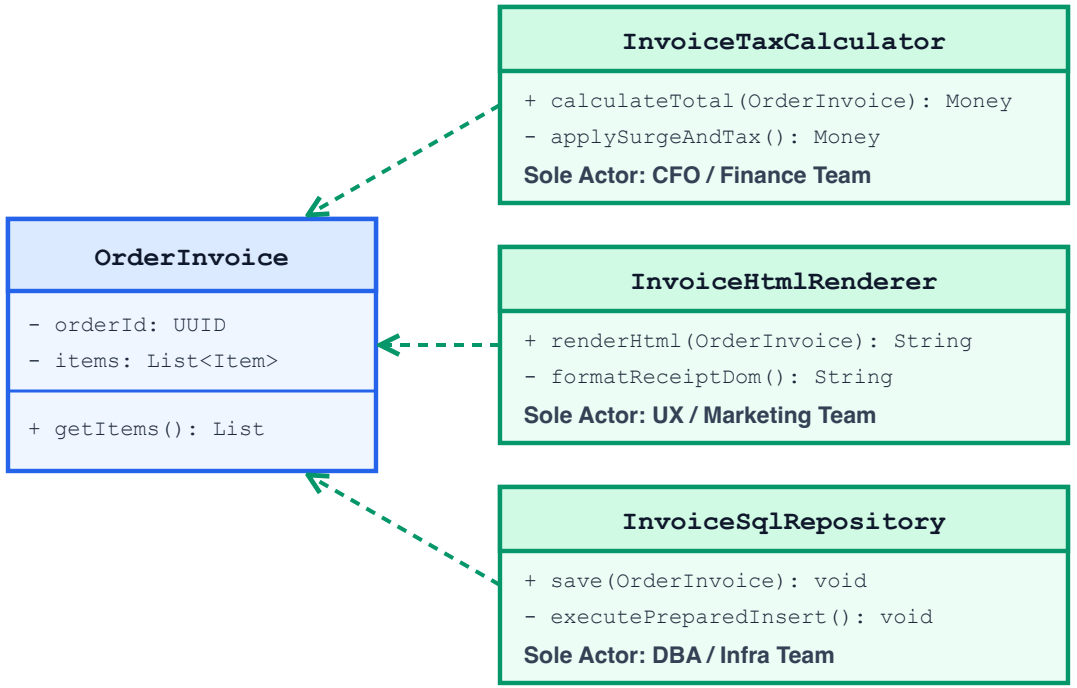
✘ Violating SRP: God Class with 3 Reasons to Change



Why This UML Design Fails (Coincidental / Logical Cohesion):

- **3 Orthogonal Axes of Change** collide inside one class file.
- A routine UI tweak by Marketing (`renderHtmlInvoice`) forces recompiling and redeploying mission-critical CFO tax calculation & SQL persistence!
- Frequent Git merge conflicts across Finance, Frontend, and Backend teams.

✓ Following SRP: 1 Cohesive Class per Actor



Why This UML Design Excels (Functional Cohesion + Data Coupling):

- **1 Class = 1 Stakeholder = 1 Reason to Change**.
- UI changes in `InvoiceHtmlRenderer` have **zero blast radius** on tax calculations or SQL persistence; each class can be unit-tested in isolation!

O — Open / Closed Principle (OCP): Design Analysis

> ***Definition (Bertrand Meyer):*** *"Software entities (classes, modules, components) should be* ***open for extension****, but* ***closed for modification****."*

❌ UML Anti-Pattern: Direct Concrete Coupling

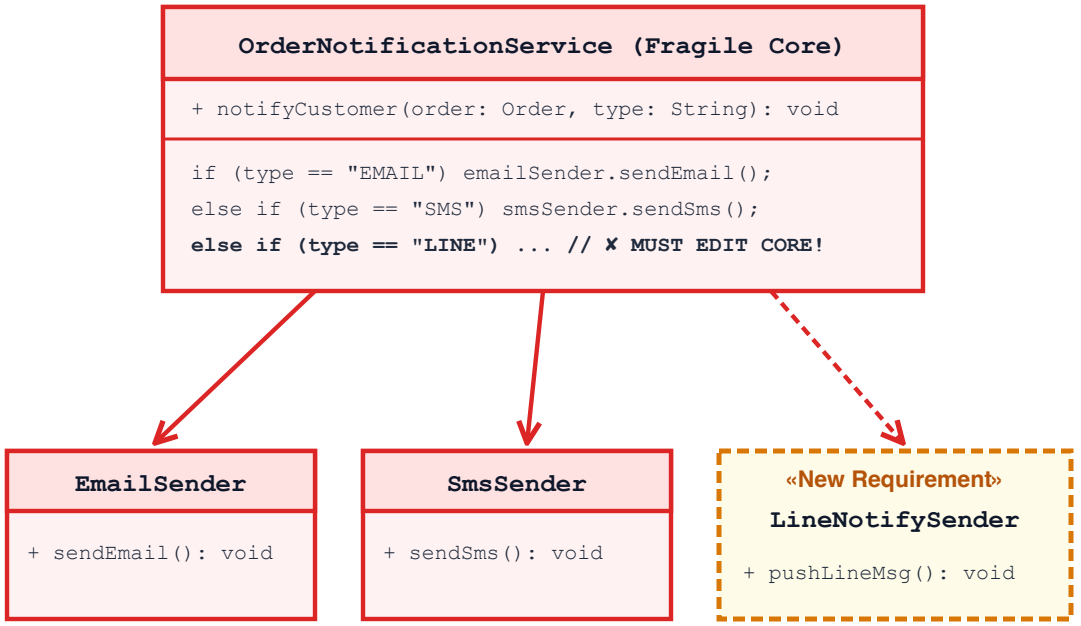
- **Hardwired Fan-Out (`——>`)**: `OrderNotificationService` directly depends on concrete `EmailSender` and `SmsSender` classes via `if / else if` type-switching.
- **Violation of "Closed for Modification"**:
  - Adding `LineNotifySender`, `WhatsApp`, or `Discord` forces engineers to **open and edit** the core `OrderNotificationService` class.
  - Every edit risks breaking existing, production-tested notification channels!

✅ UML Refactoring: Polymorphic Interface Pivot

- **Stable Abstraction (`«interface» INotificationChannel`)**:
  - `OrderNotificationService` depends **only** on `INotificationChannel.send(msg)` (🔒 **Closed for Modification**).
- **Plug-in Realization (`- - -|>`)**:
  - `EmailChannel`, `SmsChannel`, and the new `LineNotifyChannel` independently realize `INotificationChannel` (🔓 **Open for Extension**).
- **Zero-Touch Expansion**: New channels are added as standalone classes with zero impact on core orchestration.

O — Open / Closed Principle (OCP): UML Comparison

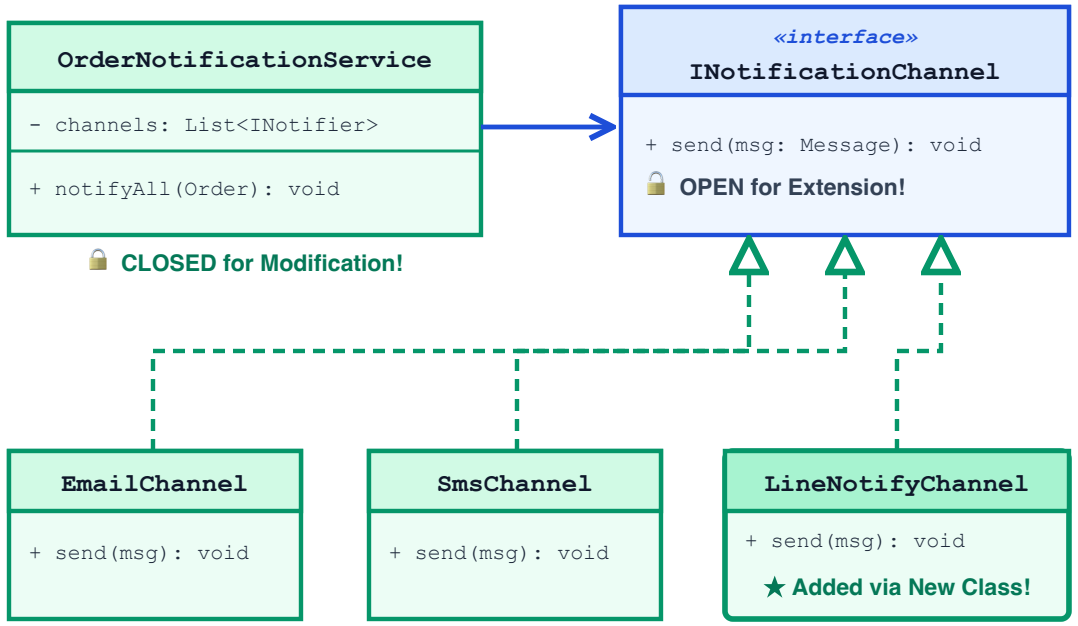
✘ Violating OCP: Rigid Concrete Coupling & Switch Chains



Why This Violates OCP (Not Closed for Modification):

- `OrderNotificationService` is directly coupled to every concrete channel.
- Every time business adds a new channel (LINE, WhatsApp, Discord), engineers must **cut open and modify** the tested core service class.
- High risk of breaking existing Email/SMS logic during routine extensions!

✓ Following OCP: Open for Extension, Closed for Modification



Why This UML Design Excels (Polymorphic Strategy / Plugin Pattern):

- `OrderNotificationService` depends solely on the stable `INotificationChannel`.
- To support **LINE Notify** or **Discord**, we simply add a new class that realizes (- - -|>) the interface—**0 lines modified** in the core service!
- Eliminates regression bugs in existing battle-tested notification channels.

L — Liskov Substitution Principle (LSP): Design Analysis

> ***Definition (Barbara Liskov):*** *"Subtypes must be behaviorally substitutable for their base types without altering any desirable correctness properties of the program."*

❌ UML Anti-Pattern: The "IS-A" Inheritance Trap

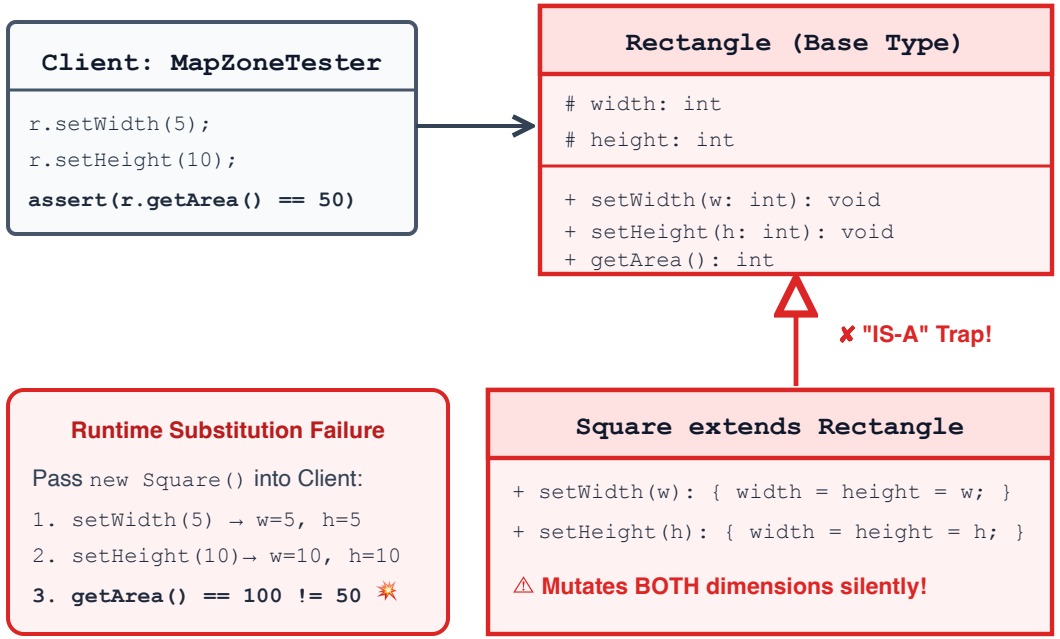
- **Broken Postcondition in `Square ——|> Rectangle`:**
  - Base `Rectangle` promises that `setHeight(10)` mutates `height` while keeping `width` invariant.
  - Subclass `Square` breaks this contract by silently mutating **both** `width` and `height` (`getArea()` yields `100 != 50`).
- **Other Common LSP Violations in Food Delivery:**
  - `CashOnDeliveryPayment ——|> OnlinePayment` throwing `UnsupportedOperationException` inside `autoRefundCard()`!

✅ UML Refactoring: Behavioral Subtyping

- **Sibling Realization (`- - -|> IShape`):**
  - Eliminate invalid inheritance between `Square` and `Rectangle`.
  - Both classes independently realize the pure contract `«interface» IShape` (`+ getArea(): int`) with immutable dimensions.
- **Design by Contract (DbC) Rules for LSP:**
  - A subtype must **never strengthen preconditions**, **never weaken postconditions**, and **never throw unexpected exceptions**.

L — Liskov Substitution Principle (LSP): UML Comparison

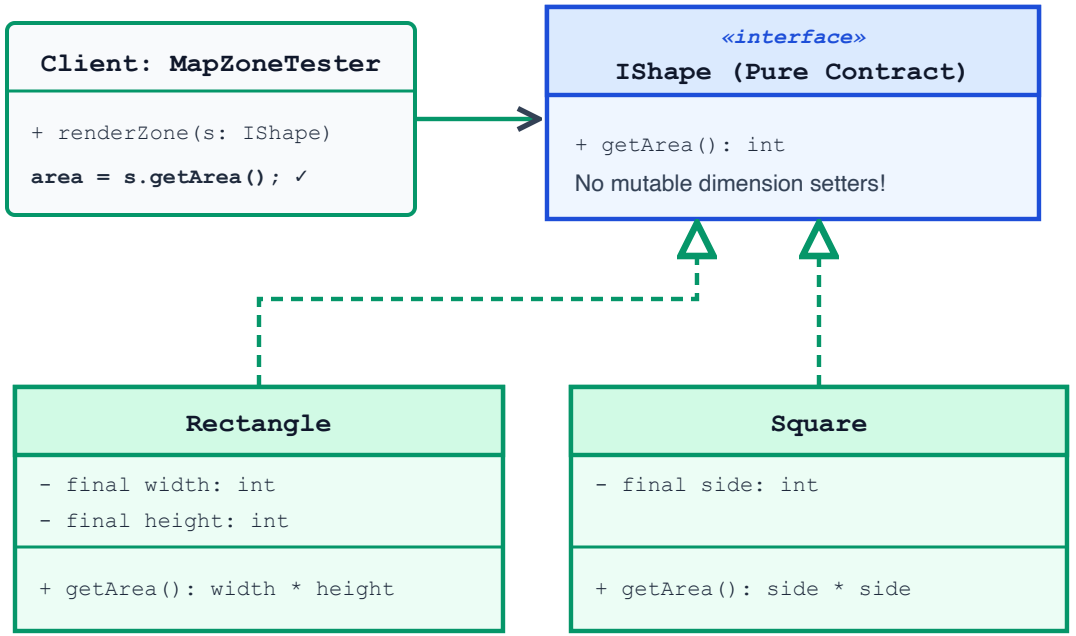
✘ Violating LSP: Subclass Breaks Base Class Postcondition



Why This Violates Liskov Substitution Principle (LSP):

- **Rectangle Postcondition Contract**: `setHeight(h)` modifies `height` while leaving `width` invariant.
- `Square` violates this postcondition! Mathematical "IS-A" does NOT imply behavioral substitutability when mutable state setters are involved.

✓ Following LSP: Substitutable Contract via Shared Abstraction



Why This UML Design Excels (100% Behavioral Substitutability):

- `Rectangle` and `Square` are independent siblings realizing `IShape`.
- Any `IShape` instance can substitute for another without hidden side effects, broken invariants, or `UnsupportedOperationException`!
- **Design by Contract Rule**: Subtypes never weaken preconditions or violate postconditions & invariants of the base contract.

I — Interface Segregation Principle (ISP): Design Analysis

> ***Definition (Robert C. Martin):*** *"Clients should not be forced to depend on interfaces or methods that they do not use."*

❌ UML Anti-Pattern: The "Fat Interface"

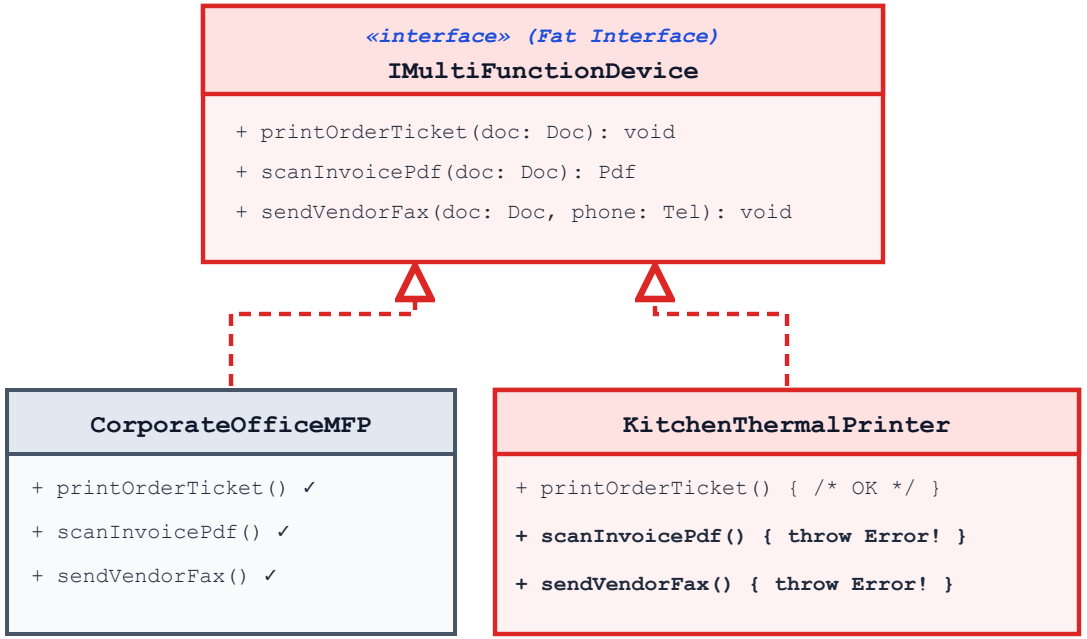
- **Polluted Contract (`IMultiFunctionDevice`)**: Bundles unrelated capabilities (`printOrderTicket`, `scanInvoicePdf`, `sendVendorFax`) into one monolithic `«interface»`.
- **Two Severe Consequences**:
  - **1. Dummy Stubs & Liskov Violations**: `KitchenThermalPrinter` must implement `scan()` and `fax()` by throwing runtime exceptions.
  - **2. Phantom Recompilation Coupling**: Changing a parameter in `sendVendorFax()` forces `KitchenThermalPrinter` to recompile!

✅ UML Refactoring: Role-Specific Interfaces

- **Granular Role Contracts**:
  - Decompose into `«interface» IPrintable`, `«interface» IScannable`, and `«interface» IFaxable`.
- **Selective Realization (`- - -|>`)**:
  - `KitchenThermalPrinter` realizes **only** `IPrintable`.
  - `CorporateOfficeMFP` composes capabilities by realizing all three (`IPrintable`, `IScannable`, `IFaxable`).
- **Result**: Zero unused methods and zero phantom coupling across unrelated clients.

I — Interface Segregation Principle (ISP): UML Comparison

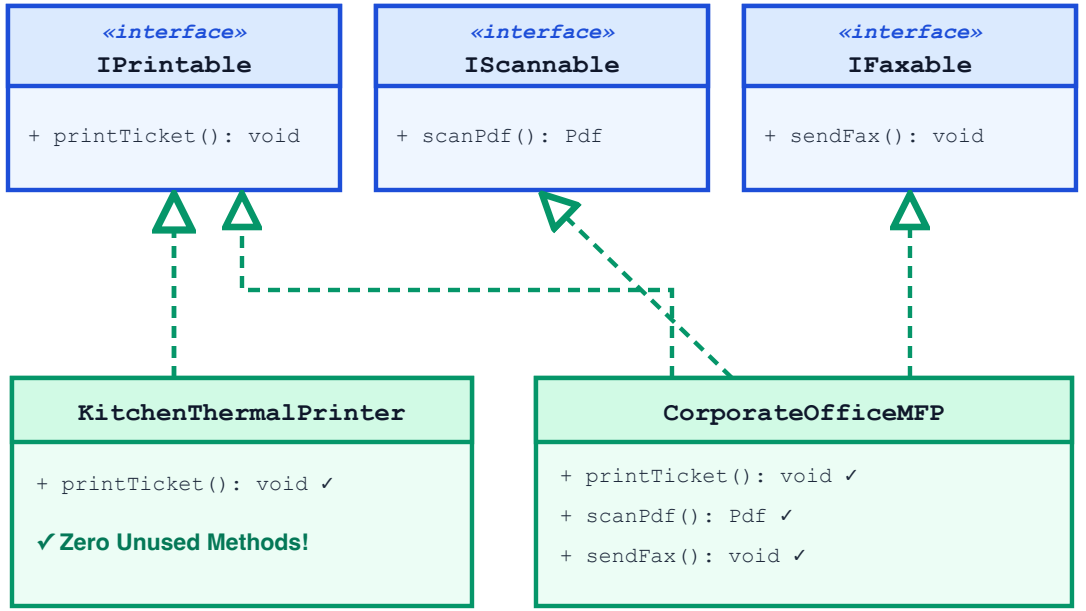
✘ Violating ISP: Bloated "Fat" Interface Pollutes Clients



Why This Violates Interface Segregation Principle (ISP):

- `KitchenThermalPrinter` only prints restaurant receipts, yet is forced to implement dummy `scan` and `fax` methods that throw runtime exceptions!
- **Phantom Coupling**: If the `Tel` parameter in `sendVendorFax()` changes, `KitchenThermalPrinter` is forced to recompile & redeploy for no reason!

✓ Following ISP: Lean, Role-Specific Interfaces



Why This UML Design Excels (Cohesive Role Contracts):

- Each interface represents a single capability (`IPrintable`, `IScannable`).
- `KitchenThermalPrinter` implements only `IPrintable`—no dummy stubs!
- Multi-function devices compose multiple interfaces via multiple realization (`implements IPrintable, IScannable, IFaxable`).

D — Dependency Inversion Principle (DIP): Design Analysis

> ***Definition (Robert C. Martin):*** *"1. High-level modules should not depend on low-level modules; both should depend on abstractions. 2. Abstractions should not depend on details; details must depend on abstractions."*

❌ UML Anti-Pattern: Downward Source Dependency

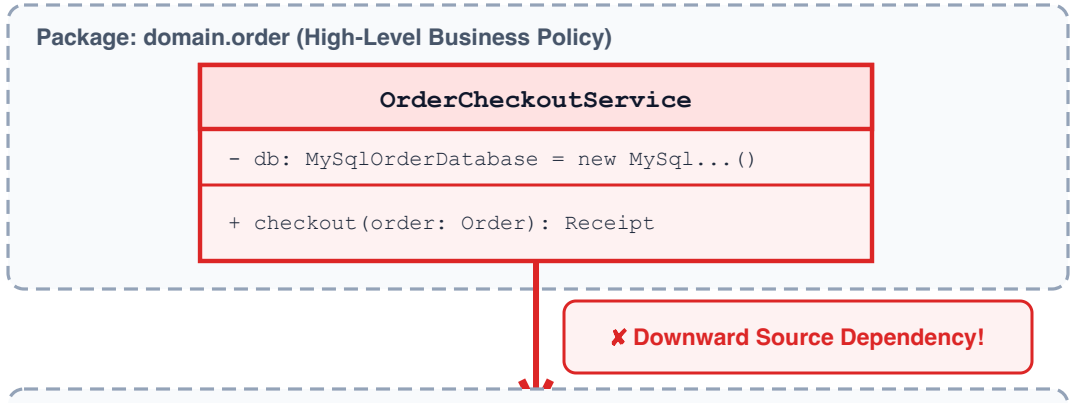
- **High-Level Bound to Low-Level (`domain ——> infra`)**:
  - `OrderCheckoutService` directly references concrete `MySqlOrderDatabase`.
- **Why This Cripples Architecture**:
  - **Immobility**: Core business policy cannot be reused with PostgreSQL without editing domain classes.
  - **Untestability**: Running unit tests on `OrderCheckoutService` requires booting a physical MySQL database server.

✅ UML Refactoring: Upward Arrow Inversion

- **Domain Owns the Interface (`IOrderRepository`)**:
  - `OrderCheckoutService` depends only on `«interface» IOrderRepository` inside `domain.order`.
- **Infrastructure Realizes Upward (`infra - - -|> domain`)**:
  - `PostgresOrderRepo` and `InMemoryMockRepo` point **upward** across the layer boundary to realize `IOrderRepository`.
- **Constructor Dependency Injection (DI)**: Inject `PostgresOrderRepo` in production and `InMemoryMockRepo` for **1ms isolated unit tests**!

D — Dependency Inversion Principle (DIP): UML Comparison

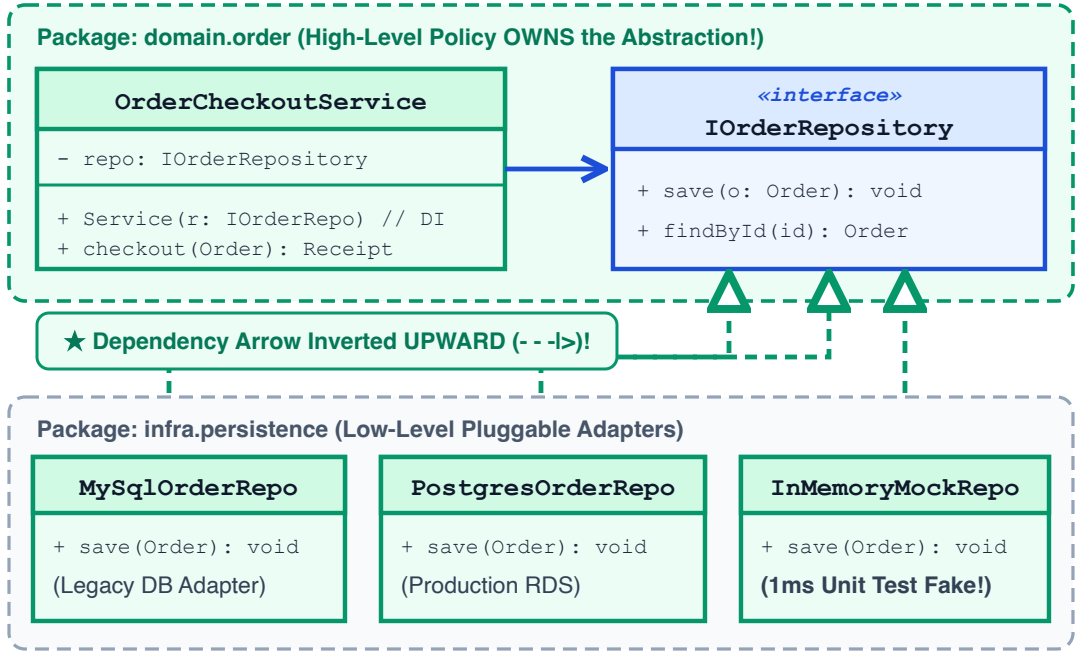
✘ Violating DIP: High-Level Policy Coupled to Low-Level Detail



Why This Violates Dependency Inversion Principle (DIP):

- **High-level business policy** directly imports low-level MySQL JDBC details.
- **Immobility**: Cannot reuse `OrderCheckoutService` with PostgreSQL or DynamoDB without rewriting core business classes.
- **Untestable**: Cannot run fast unit tests without booting a real MySQL DB!

✓ Following DIP: Both Layers Depend on Domain Abstraction



Why This UML Design Excels (Inversion of Control & Clean Architecture):

- **Rule 1**: High-level `OrderCheckoutService` and low-level `PostgresOrderRepo` both depend on the `IOrderRepository` abstraction.
- **Rule 2**: Low-level DB adapters point **UPWARD** into the Domain package; injected via Constructor Dependency Injection (DI) for 1ms unit testing!

Peer Discussion: Exploring the GoF Observer Pattern (Motivation & Structure)

Web Search & Pair Analysis: The GoF Observer Design Pattern (4 Mins)

- **1. Search Online:** Look up the **Observer Design Pattern** (e.g., on *Refactoring.Guru* or *GoF Catalog*).
- **2. Design Motivation (What problem does it solve?):**
  - When one **Subject** changes state (e.g., a Food Delivery `Order` transitions to `READY_FOR_PICKUP`), multiple **Dependents** (`CustomerApp`, `RiderApp`, `MerchantDisplay`) must update automatically—**without hardcoding concrete class calls inside `Order`**!
- **3. UML Structure & SOLID Mapping (Discuss with Your Partner):**
  - Sketch the 4 UML roles: `Subject (Publisher)`, `«interface» IObserver (Subscriber)`, and `ConcreteObserverA / B`.
  - How does the Observer Pattern embody **OCP** (adding new subscribers without editing `Subject`) and **DIP** (`Subject` depends only on `«interface» IObserver`)?



Concept Check Question 4 (CCQ 4)

Which SOLID object-oriented design principle explicitly advocates that when introducing new system capabilities, developers should extend functionality by adding new classes rather than modifying existing, battle-tested code?

- **A.** Single Responsibility Principle (SRP)
- **B.** Open / Closed Principle (OCP)
- **C.** Interface Segregation Principle (ISP)
- **D.** Liskov Substitution Principle (LSP)



5.6 Stage 4 — Data & Persistence Design: ERD, Caching & Polyglot Storage

> *"Bad programmers worry about the code. Good programmers worry about data structures and their relationships."*
> *— Linus Torvalds (Creator of Linux & Git)*

1. Three-Level Data Modeling, ERD Notation & Normalization

> ***Stage 4 Modeling Purpose:*** *Translates domain objects into* ***integrity-enforced relational schemas*** *and* ***high-speed physical indexes***.

3-Level Data Modeling & UML ERD Notation

- **1. Conceptual → 2. Logical ERD → 3. Physical Schema**:
  - Progresses from business concepts to normalized table columns, and finally to DBMS-specific indexes and partitions.
- **Core UML Data / ERD Notations**:
  - `«PK»` **(Primary Key)**: Unique immutable identifier (`order_id: UUID`).
  - `«FK»` **(Foreign Key)**: Referential integrity link (`customer_id` referencing `customers.customer_id`).
  - **Multiplicity (`1 — 0..*`, `1 ◆— 1..*`)**: Distinguishes optional associations from lifecycle-bound composition (`Order` ◆—— `OrderItem`).

3NF Normalization vs. Physical Optimization

- **Third Normal Form (3NF — Write Integrity)**:
  - Every non-key column depends *only* on the Primary Key (e.g., store `restaurant_id` on `orders`, never duplicate restaurant address strings across millions of orders).
  - **Benefit**: Eliminates update anomalies and guarantees single source of truth.
- **Physical Indexing & Partitioning (Read Speed)**:
  - **Composite Index (`«idx»`)**: `(customer_id, created_at DESC)` accelerates "*My Order History*" from `800ms` full-table scan to `< 2ms` B-Tree lookup.
  - **Table Partitioning**: Partitioning `orders` monthly by `created_at` keeps active working sets small in RAM.

2. Polyglot Persistence, Caching & Distributed Consistency

> ***Architectural Reality:*** *Modern cloud systems pair* ***ACID Relational Databases*** *with* ***In-Memory Caches*** *and* ***Event Logs*** *(Polyglot Persistence).*

Matching Storage Engines to Workloads

- **1. Relational OLTP (`PostgreSQL` — ACID)**:
  - *Food Delivery Use*: `orders`, `order_items`, `payments`.
  - *Design Rule*: Use `«UQ» idempotency_key` and row-level locking to guarantee zero double-charging.
- **2. In-Memory Key-Value / Spatial (`Redis` — Ephemeral)**:
  - *Food Delivery Use*: Live rider GPS (`rider:geo:{city}`) & restaurant menu **Cache-Aside** (`TTL = 5 min`).
  - *Design Rule*: High write/read velocity (`< 5ms`), auto-evicted via `TTL = 30s` if a rider goes offline.
- **3. Append-Only Event Log (`Kafka` — Stream)**:
  - *Food Delivery Use*: High-throughput immutable audit log of domain events partitioned by `order_id`.

Managing Consistency: ACID vs. BASE & Outbox

- **Inside a Single Service (`ACID`)**:
  - Atomicity, Consistency, Isolation, Durability via local SQL transactions.
- **Across Distributed Services (`BASE` & Saga)**:
  - *Problem*: How to commit an `Order` in PostgreSQL and publish `OrderPlacedEvent` to Kafka without dual-write data loss?
  - *Solution — **Transactional Outbox Pattern***:
    1. Insert `orders` row AND `outbox_events` row in the **same local ACID transaction**.
    2. A background relay publishes `outbox_events` to Kafka (`At-Least-Once`), while downstream consumers enforce idempotency!

3-Level Data Modeling, ERD & Polyglot Storage in Food Delivery

Stage 4: Data & Persistence Design — 3-Level Modeling, ERD & Polyglot Storage

Designing Relational Schemas (ACID), Ephemeral Spatial Caches (TTL), and Event Logs for the Food Delivery System

1. Three-Level Data Modeling Pipeline

LEVEL 1: CONCEPTUAL DATA MODEL (Domain View)

- High-level business entities & relationships (technology-agnostic).

`[Customer] 1 ——— places ——— 0..* [Order] 1 ——— contains ——— 1..*`

Audience: Domain Experts, Product Owners & Architects.

LEVEL 2: LOGICAL ERD MODEL (Schema Structure)

- Defines attributes, data types, Primary Keys (PK), Foreign Keys (FK).
- Applies Normalization (1NF → 2NF → 3NF) to eliminate write anomalies.

`«PK» order_id: UUID | «FK» customer_id: UUID | status: VARCHAR`

Audience: Software Engineers & Data Architects.

LEVEL 3: PHYSICAL DATA SCHEMA (DBMS Optimization)

- SQL DDL, B-Tree / Geo Indexes, Table Partitioning & Sharding keys.
- Read Replicas, Redis Cache-Aside TTLs, and Kafka topic retention.

```
CREATE INDEX idx_orders_cust_created ON orders(customer_id, created_
```

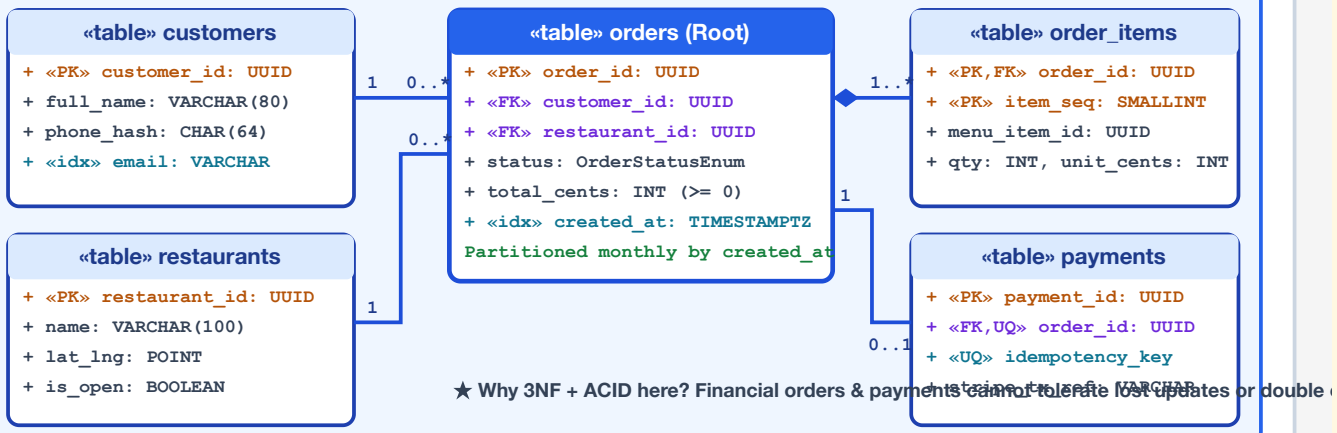
Audience: DBAs, Backend Engineers & SREs.

UML Data / ERD Notation Key:

«PK»	Primary Key (Unique row identifier, e.g., order_id)
«FK»	Foreign Key (References PK of parent entity table)
1 ——— 0..*	One-to-Zero-or-Many (A Customer has 0 or many Orders)
1 ◆—— 1..*	Composition: Order owns 1..* OrderItems (lifecycle bound)
«idx»	Database Index accelerating high-frequency WHERE / JOIN queries

2. Food Delivery System: Logical ERD & Polyglot Persistence Architecture

A. Relational OLTP Store (PostgreSQL — Strict ACID Transactions & 3NF Normalization)



★ Why 3NF + ACID here? Financial orders & payments cannot tolerate lost updates or double

B. In-Memory Spatial Store (Redis Cluster)

```
Key Pattern: rider:geo:{city_id}
Structure: Sorted Set (64-bit GeoHash)
Value: {rider_id, lat, lng, status}
Expiration: TTL = 30 seconds (Auto-Evict)
```

Why Not SQL for Rider GPS?

- 50k riders ping every 5s (10,000 writes/sec).
- Redis GEORADIUS finds nearest riders in < 5ms without B-Tree index lock contention!

C. Immutable Event Log (Apache Kafka)

```
Topic: food.orders.domain-events
Partition Key: order_id (Strict ordering)
Schema: Avro / Protobuf (Versioned)
Consistency: Transactional Outbox + Saga
```

Polyglot Data Synchronization:

- Order commits to PostgreSQL → Outbox emits event to Kafka → Dispatch, Analytics & Read Projections consume asynchronously (BASE).

Concept Check Question 5 (CCQ 5)

In Stage 4 (**Data & Persistence Design**), why does a Food Delivery Platform store `orders` and `payments` in **PostgreSQL (Relational OLTP)** while storing 50,000 real-time courier GPS coordinates in **Redis (`GeoHash` + `TTL = 30s`)**?

- **A.** PostgreSQL lacks primary key support for tables, whereas Redis enforces Third Normal Form (3NF) foreign keys.
- **B.** Redis archives historical financial ledgers on disk, whereas PostgreSQL evicts all unpaid orders after 30 seconds.
- **C.** PostgreSQL ensures ACID financial integrity, while in-memory Redis serves high-frequency GPS queries in `< 5ms`.
- **D.** Writing live courier GPS streams into PostgreSQL eliminates all wireless network latency on mobile client devices.

5.7 AI-Augmented Software Design

> *"AI will not replace software architects, but architects who use AI—while grounded in solid engineering principles—will replace those who do not."*

AI as a Copilot Across the 4 Stages of Software Design

- **The "Copilot, Not Autopilot" Paradigm:**
  - AI excels at **generating candidate models, drafting contracts, detecting smells, and writing schemas**; human architects own **context, budget, security, and accountability**.
- **4 High-Impact Applications Mapped to Our 4 Design Stages:**
  - **Stage 1 (Arch Design — 5.3)**: Simulating **ATAM failure scenarios** ("*What happens if Redis crashes during dinner surge?*"), drafting **ADRs**, and generating **UML Package/Deployment** PlantUML code.
  - **Stage 2 (Interface Spec — 5.4)**: Generating **OpenAPI 3.1 / Protobuf** contracts and deriving **DbC Preconditions, Postconditions, and boundary test cases**.
  - **Stage 3 (OO Design — 5.5)**: Scanning for God Classes / cyclic dependencies and refactoring `switch/case` blocks into **SOLID / GoF Strategy** patterns.
  - **Stage 4 (Data Design — 5.6)**: Translating domain models into **3NF SQL DDL**, recommending composite **B-Tree indexes**, and spotting `N+1` query bottlenecks.

Human-in-the-Loop: Mitigating AI Blind Spots

- **Risks of Unchecked AI Adoption:**
  - **Premature Distributed Complexity:** AI frequently over-engineers, recommending microservices, Kubernetes, and Kafka for simple internal tools that 3 engineers could finish in a fortnight with a monolith.
  - **Context Blindness:** AI is blind to your team's tech-stack familiarity, cloud hosting budgets, and internal security compliance constraints.
  - **Superficial Refactoring:** Renaming variables without resolving fundamental architectural bottlenecks or data coupling.
- **The Golden Rule of System Architecture:**

> ***"AI Proposes Trade-offs; Lead Architects Make Decisions!"***
> *Human engineers must evaluate organizational context and bear ultimate responsibility for system viability.*

5.8 Conceptual Synthesis & References

> *"Simplicity is prerequisite for reliability."*
> *— Edsger W. Dijkstra*

Core Curriculum Self-Assessment & Key Takeaways

Chapter Section	Design Stage & UML / Model	Golden Engineering Heuristic
5.1 Design Process & Specs	4-Stage Workflow, **SDD & ADRs**	Catch flaws early ($1 vs. $100); document the *How* (SDD) and *Why* (ADR).
5.2 Universal Principles	**Cohesion & Coupling** (UML Compare)	**Maximize Cohesion, Minimize Coupling!** Hide internal secrets.
5.3 Stage 1: Arch Design	6 Patterns, ATAM, **Package & Deploy UML**	Synthesize patterns; enforce acyclic imports & cloud fault isolation.
5.4 Stage 2: Interface Spec	**Design by Contract (DbC) & Component UML**	Specify `Pre`, `Post` & `Invariant`; wire via Lollipop (`—o`) & Socket (`—(`).
5.5 Stage 3: OO Design	GoF 23, **5 SOLID & Class UML**	Depend on abstractions (DIP); subclasses must honor DbC contracts (LSP).
5.6 Stage 4: Data Design	**3-Level ERD**, 3NF & Polyglot Storage	Use ACID SQL for money, Redis TTL for live Geo-cache, Outbox for Kafka.
5.7 AI in Design	Copilot Across Stages 1–4	AI proposes options; human architects own trade-offs & context!

Classic Literature & Essential References

- **Foundational Software Architecture & Design Specifications:**
  - Len Bass, Paul Clements, Rick Kazman (SEI): *Software Architecture in Practice*, 4th Edition, Addison-Wesley.
  - Bertrand Meyer: *Object-Oriented Software Construction* (Origin of Design by Contract — DbC), Prentice Hall.
  - IEEE Std 1016: *IEEE Standard for Information Technology—Systems Design—Software Design Descriptions (SDD)*.
  - Michael Nygard: *Documenting Architecture Decisions (ADR)*, 2011.
- **Classic Design Principles, UML, Data & Patterns Literature:**
  - Robert C. Martin (Uncle Bob): *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall.
  - Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software (GoF)*, Addison-Wesley.
  - Martin Fowler: *UML Distilled* & *Patterns of Enterprise Application Architecture*, Addison-Wesley.
  - Martin Kleppmann: *Designing Data-Intensive Applications (DDIA)*, O'Reilly Media.

Appendix A: Suggested Answers for CCQ 1 – CCQ 5

Question	Ans	Core Design Principle / UML Rule & Concise Rationale
CCQ 1 (Sec 5.2) *Cohesion & Coupling*	**B**	**Low Cohesion & Content Coupling**: `OrderReportUtils` mixes unrelated tasks (**Logical/Coincidental Cohesion**), while modifying `rawSqlString` directly violates information hiding (**Content Coupling**). Refactor into single-purpose classes communicating via **Data Coupling** (DTOs).
CCQ 2 (Sec 5.3) *Macro UML Diagrams*	**C**	**Compile-Time vs. Runtime Topology**: **UML Package Diagrams** organize compile-time source namespaces and acyclic `«import»` rules; **UML Deployment Diagrams** map runtime 3D physical/cloud nodes, execution environments, and deployed `«artifact»` binaries.
CCQ 3 (Sec 5.4) *DbC & Component UML*	**B**	**Rights & Obligations Matrix**: The **Caller (Client)** must satisfy the **Precondition** (`qty > 0`), while the **Supplier** guarantees the **Postcondition** (`OrderConfirmation`), exposed as a **Lollipop (`—○`)** wired into the caller's **Socket (`—(`)**.
CCQ 4 (Sec 5.5) *SOLID OO Principles*	**A**	**DIP & LSP Violations**: Instantiating `new MySqlOrderDatabase()` inside domain logic violates **DIP** (depend on abstractions); throwing `UnsupportedOperationException` on `refund()` violates **LSP** (subtypes must honor parent postconditions) and **ISP**.
CCQ 5 (Sec 5.6) *Data & Polyglot DBs*	**C**	**Workload-Matched Polyglot Persistence**: **PostgreSQL (OLTP)** guarantees **ACID** & 3NF integrity for financial orders/payments, whereas **Redis (`GeoHash` + `TTL = 30s`)** serves 10,000/sec ephemeral courier GPS lookups in `< 5ms` without B-Tree lock contention.

Appendix B: Solution — Peer Discussion 5.3 (Chess MVC Package UML)

> ***Reference Solution:*** *A clean 3-package MVC separation where* `domain.model` *has* ***zero upward dependencies*** *on* `ui.view` *or* `ui.controller`*.*

1. Reference PlantUML Package Diagram

```
@startuml
skinparam packageStyle folder

package "ui.controller" {
  [GameController]
  [MoveInputHandler]
}
package "ui.view" {
  [ChessBoardGUI]
  [MoveHistoryPanel]
}
package "domain.model" {
  [BoardState]
  [PieceRulesEngine]
  [MoveValidator]
}

"ui.controller" ..> "domain.model" : «import» mutates/validates
"ui.controller" ..> "ui.view"      : «import» updates UI
"ui.view"       ..> "domain.model" : «import» reads state
@enduml
```

2. Key Architectural Design Insights

- **1. Package Classification**:
  - `domain.model`: `BoardState`, `PieceRulesEngine`, `MoveValidator` (pure chess rules & state).
  - `ui.view`: `ChessBoardGUI`, `MoveHistoryPanel` (rendering only).
  - `ui.controller`: `GameController`, `MoveInputHandler` (translates clicks/drags into domain commands).
- **2. Acyclic Dependency Rule (ADP)**:
  - Arrows flow strictly **toward** `domain.model`.
  - **Forbidden Import**: `domain.model ..> ui.view` is strictly banned! Because `domain.model` imports nothing from `ui.*`, you can run 10,000 automated JUnit chess tests in 1 second or swap `ChessBoardGUI` for a Web/Mobile UI without touching a single line of chess logic!

Appendix C: Solution — Peer Discussion 5.4 (`bubbleSort` Javadoc Spec)

> ***Reference Solution:*** *A complete sorting specification requires* ***both*** *the* ***Ordered Postcondition*** *AND the* ***Permutation Postcondition****!*

1. Contract-Enhanced Javadoc & JML Spec

```
/**
 * Sorts the specified array in-place into ascending numerical order.
 *
 * @param arr the integer array to be sorted (mutated in-place)
 * @throws IllegalArgumentException if {@code arr == null}
 *
 * @pre  arr != null (Empty or 1-element arrays are valid no-ops)
 * @post 1. Length Preserved: arr.length == \old(arr.length)
 * @post 2. Ordered: \forall int i; 0 <= i < arr.length - 1;
 *                   arr[i] <= arr[i + 1]
 * @post 3. Permutation (Multiset Conservation):
 *          arr is a permutation of \old(arr) — every element's
 *          frequency count is identical before and after sorting.
 * @invariant Outer Loop (pass k): The last k elements
 *          arr[n - k .. n - 1] are in their final sorted positions.
 */
public static void bubbleSort(int[] arr) { ... }
```

2. Discussion Answers & Contract Analysis

- **Q1. Precondition (`@param` / `@throws`):**
  - `arr != null`. If `arr == null`, throw `IllegalArgumentException`. Note that `arr.length <= 1` is valid and returns immediately.
- **Q2 & Q3. Why "Ordered" Alone Is Broken!:**
  - Suppose a lazy developer implements `bubbleSort(arr)` as: `Arrays.fill(arr, 0);` (overwriting every element with `0`).
  - Does `[0, 0, 0, 0]` satisfy `arr[i] <= arr[i+1]`? **YES!**
  - Therefore, a formal sorting postcondition **must** specify two clauses:
    1. **Ordered**: `arr[i] <= arr[i + 1]` for all adjacent pairs.
    2. **Permutation**: The output multiset equals `\old(arr)` (no elements fabricated, overwritten, or lost!).

Appendix D: Solution — Peer Discussion 5.5 (GoF Observer Pattern)

> ***Reference Solution:*** *The* ***Observer Pattern*** *decouples a state-changing* ***Subject*** *from an open-ended set of* ***Observers*** *via a polymorphic notification interface.*

1. Motivation & UML Class Structure

- **Design Motivation (The Polling vs. Coupling Dilemma)**:
  - Without Observer, either clients waste CPU polling `getOrderStatus()` every second, OR `OrderService` hardcodes calls to `CustomerApp`, `KitchenDisplay`, and `RiderTracker` (violating **SRP & OCP**).
- **GoF UML Participants**:
  - **`Subject`** (**`OrderEventPublisher`**): Maintains `List<OrderObserver>` with `+ attach(o)`, `+ detach(o)`, and `+ notifyObservers(event)`.
  - **`«interface» OrderObserver`**: Declares `+ onOrderStateChanged(OrderEvent e)`.
  - **`ConcreteObservers`**: `CustomerPushObserver`, `KitchenScreenObserver`, `RiderMapObserver` implement `OrderObserver` (`..|>`).

2. SOLID Principles & Food Delivery Application

```
@startuml
class OrderService {
  - observers: List<OrderObserver>
  + attach(o: OrderObserver)
  + updateStatus(id, status)
}
interface OrderObserver <<interface>> {
  + onOrderStateChanged(e: OrderEvent)
}
OrderService o--> "*" OrderObserver : notifies >
OrderObserver <|.. CustomerPushObserver
OrderObserver <|.. KitchenScreenObserver
OrderObserver <|.. FraudAuditObserver
@enduml
```

- **How Observer Embodies OCP & DIP**:
  - **DIP**: High-level `OrderService` depends only on `«interface» OrderObserver`, never on concrete notification channels.
  - **OCP**: To add a new `FraudAuditObserver` next month, we simply write 1 new class and call `attach()` — **zero lines of `OrderService` are modified!**

Thank You for Listening!

Questions & Discussion

Prof. Nien-Lin Hsueh
Department of Information Engineering and Computer Science
Feng Chia University